Graduation Project Report

Department of Software Engineering
Faculty of Information Technology
Al-Ahliyya Amman University

RukhsTy

A project submitted
in partial fulfillment of the requirements for the degree of
Bachelor in Software Engineering

by:

Hatm Sinjlawi | 202110267
Salma Daghistani | 202220327
Omar Albow | 202411843
Shahd Taha | 202421114

Supervised by:

Dr. Ahmad Shubita

Month, Year

Undertaking

This is to declare that the project entitled **"RukhsTy"** is an original work carried out by the undersigned students in partial fulfillment of the requirements for the degree of **Bachelor in Software Engineering** at the Department of Software Engineering, Faculty of Information Technology, Al-Ahliyya Amman University.
All analysis, design, implementation, testing, and documentation activities related to this project have been completed by the undersigned project members. Furthermore, this project has not been submitted, in whole or in part, to any other college or university for the award of any academic degree.

Name and Signature:…… Date: ……

Name and Signature:…… Date: ……

Name and Signature:…… Date: ……

Name and Signature:…… Date: ……

Rukhsty — Acknowledgement

First and foremost, we praise and thank Allah (SWT) for granting us the ability, patience, and determination to complete this graduation project successfully.

We would like to express our sincere gratitude to Al-Ahliyya Amman University, the Faculty of Information Technology, and the Department of Software Engineering for providing an academic environment that encouraged learning, research, and the practical application of software engineering principles.

We are deeply thankful to our supervisor, **Dr. Ahmad Shubita**, for his continuous guidance, valuable feedback, and professional support throughout all phases of this project, from planning and analysis to implementation and documentation. His supervision helped us improve the quality of our work and stay focused on achieving the project objectives.

We would also like to extend our appreciation to the Traffic Department of Jordan and the related stakeholders for providing valuable domain knowledge and practical insights. Their input helped us better understand real-world driving license procedures and translate them into functional system requirements for **RukhsTy**.

Finally, we are sincerely grateful to our families and friends for their continuous encouragement, understanding, and support during the project period. Their motivation and patience played an important role in helping us overcome challenges and complete this work successfully.

RukhsTy-Abstract

RukhsTy is a full-stack web platform designed to digitize and simplify the first-time driving license process in Jordan. The current process is often manual, paper-based, and dependent on repeated in-person visits, which may cause long waiting times, duplicated document handling, and limited visibility into application status.

The proposed system provides an end-to-end digital workflow that allows citizens to register, submit license applications, upload required documents, book appointments, track application progress, complete mock eFAWATEERcom payment, view a digital driving license, and request mock Aramex delivery. The platform also supports role-based dashboards for administrators and officers, including security, medical, theory, and practical officers, so each stage of the licensing process can be reviewed and managed clearly.

RukhsTy was developed using React, TypeScript, and Tailwind CSS for the frontend, Node.js and Express.js for the backend, and PostgreSQL with Drizzle ORM for database management. Authentication and authorization are handled using JWT and role-based access control.

The main outcome of the project is a practical digital solution that improves transparency, reduces paperwork, minimizes unnecessary physical visits, and demonstrates how public service procedures can be transformed into an organized, secure, and user-friendly digital workflow.

Table of Contents

Chapter 1: Introduction **1**
1.1 Overview 1
1.2 Project Motivation 2
1.3 Review of Related Literature and Systems 3
1.4 Problem Statement 5
1.5 Project Objectives 6
1.6 Project Plan 7
Chapter 2: The Development Process **9**
Chapter 3: Requirements Engineering and Analysis **10**
3.1 Stakeholders 10
3.2 Functional Requirements 12
3.3 Non-Functional Requirements 13
3.4 Use Case Diagram 14
Chapter 4: Architecture and Design **20**
4.1 Software Architecture 20
4.2 Software Design 22
4.2.1 Sequence Diagrams 22
4.2.2 Class Diagram 27
4.2.3 State Transition Diagram 28
4.2.4 ER Diagram 29
4.2.5 Activity Diagram 30
4.3 User Interface Design 31
Chapter 5: System Implementation **42**
5.1 Overview 42
5.2 Tools and Technologies Used 43
5.2 Tools and Technologies Summary 44
5.3 Important Code Fragments 45
Chapter 6: Testing and Validation **56**
6.1 Black-Box 56
6.2 White-Box 59
6.3 Testing Automation 61
Chapter 7: Conclusion and Future Work **63**
7.1 Conclusion 63
7.2 Future Work 64
References **65**
Appendix **66**

List of Figures

Figure 1. Use Case Diagram 14
Figure 2. Register Account Sequence Diagram 22
Figure 3. Submit Application Sequence Diagram 22
Figure 4. Track Status Sequence Diagram 23
Figure 5. Book Appointment Sequence Diagram 23
Figure 6. Security Approval Sequence Diagram 24
Figure 7. Manage Medical Test Sequence Diagram 24
Figure 8. Manage Theory Exam Sequence Diagram 25
Figure 9. Manage Practical Exam Sequence Diagram 25
Figure 10. Pay by eFAWATEERcom Sequence Diagram 26
Figure 11. Class Diagram 27
Figure 12. State Transition Diagram 28
Figure 13. ER Diagram 29
Figure 14. Activity Diagram 30
Figure 15. RukhsTy Home Page 31
Figure 16. RukhsTy Services Page 31
Figure 17. Register Page 32
Figure 18. Login Page 32
Figure 19. User Dashboard 33
Figure 20. User Services Page 33
Figure 21. My Appointments Page 34
Figure 22. Digital Driving License Page 34
Figure 23. User Profile Page 35
Figure 24. Upload Training Certificate 35
Figure 25. Choose License Category 36
Figure 26. Confirm Personal Information 36
Figure 27. Submit for Security Review 37
Figure 28. Application Tracking Page 37
Figure 29. Security Review Dashboard 38
Figure 30. Book Medical / Vision Test 38
Figure 31. Medical / Vision Test Result 39
Figure 32. Book Theory Exam 39
Figure 33. Theory Exam Verification 40
Figure 34. Practical Exam Result 40
Figure 35. License Issued Dashboard 41
Figure 36. Aramex Delivery Page 41
Figure 37. Authentication and Login Backend 45
Figure 38. Frontend Login and Role Redirection 46
Figure 39. User Registration Backend 47
Figure 40. Database Schema / Model Layer 48
Figure 41. Submit First-Time Driving License Application 49
Figure 42. Security Officer Workflow 50
Figure 43. Medical Officer Workflow 51
Figure 44. Digital License Issuing 52
Figure 45. License Issuance Eligibility and Duplicate Prevention 53

Figure 46. Mock eFAWATEERcom Payment 54
Figure 47. Frontend Route Protection and Component Structure 55

List of Tables

Table 1 viii
Table 2 12
Table 3 13
Table 4 44
Table 5 58
Table 6 60
Table 7 62

List of Abbreviations

Abbreviation	Stands For
AAU	Al-Ahliyya Amman University
API	Application Programming Interface
DB	Database
DVLD	Driver and Vehicle Licensing Department
ERD	Entity Relationship Diagram
GUI	Graphical User Interface
JWT	JSON Web Token
ORM	Object Relational Mapping
QR	Quick Response
REST	Representational State Transfer
RTL	Right-To-Left
UI	User Interface
UX	User Experience

Table 1

Chapter 1: Introduction

1.1 Overview

Rukhsty is a full-stack web-based platform designed to modernize and simplify the driving license service process in Jordan. Driving license services are considered essential public services because many citizens depend on private vehicles for work, education, transportation, and daily activities. However, the traditional licensing process often requires several in-person visits, manual paperwork, repeated document submission, and direct follow-up with different service departments.

In many cases, applicants need to visit traffic offices to submit documents, open files, schedule required tests, follow up on their application status, complete approvals, and receive the final license. These steps can lead to long waiting times, unclear procedures, duplicated data entry, lost documents, and limited visibility into the current status of the application. From an administrative perspective, manual workflows also increase pressure on employees and make it more difficult to monitor applications efficiently.

Rukhsty addresses these challenges by providing a digital platform that organizes the driving license workflow in one system. The platform allows citizens to register, log in, submit license applications, upload required documents such as training certificates, track application progress, book appointments, complete mock payment, view the issued digital license, verify the license using QR code, and request optional delivery.

The system also supports multiple officer roles involved in the licensing process. These roles include Security Officer, Medical Officer, Theory Officer, and Practical Officer. Each officer has a dedicated dashboard and specific responsibilities within the application workflow. The Security Officer reviews applications and approves or rejects them, while the Medical, Theory, and Practical Officers manage test stages and record results. In addition, the Admin manages users, applications, system data, and reports.

The scope of Rukhsty is to provide a complete digital driving license management system for academic demonstration. It focuses on improving accessibility, transparency, organization, and efficiency in the licensing workflow. The system uses safe mock integrations for eFAWATEERcom payment and Aramex delivery to demonstrate real-world service flow without connecting to real external APIs.

1.2 Project Motivation

The Rukhsty project was selected because the driving license process is a common service used by a large number of citizens in Jordan. Many applicants experience difficulties such as long queues, repeated visits, unclear application progress, and manual document handling. These challenges make the process time-consuming and stressful for citizens, while also increasing the workload on officers and administrative employees.

As software engineering students, we selected this project because it represents a real-life problem that can be improved through digital transformation. The licensing process contains several stages that can be modeled, automated, tracked, and managed using software engineering principles. This made the project suitable for applying important concepts such as requirements analysis, system design, database modeling, role-based access control, API development, testing, and documentation.

The importance of Rukhsty comes from its ability to provide a unified digital platform for both citizens and officers. Citizens can complete many steps online instead of depending on repeated physical visits. Officers can manage applications more clearly through role-based dashboards. The Admin can monitor users, applications, and reports from one place. This improves transparency, reduces paperwork, and supports a more organized workflow.

The new idea introduced by Rukhsty is the combination of the complete driving license process into one digital journey. Instead of separating application submission, officer review, payment, license issuing, and delivery into disconnected steps, Rukhsty connects these services inside one platform. The system also includes a mock eFAWATEERcom payment flow, a digital license card with QR verification, and a mock Aramex delivery request to demonstrate a realistic end-to-end licensing experience

1.3 Review of Related Literature and Systems

Digital government has become an important direction for improving public service delivery, reducing administrative burden, and increasing transparency between citizens and public institutions. The World Bank describes digital public infrastructure as a foundation for trusted and scalable digital services, including digital identity, interoperable payment platforms, and secure data exchange [1]. Similarly, OECD digital government studies emphasize that digital transformation can help governments become more agile, transparent, inclusive, and responsive to citizens' needs [2]. These ideas are directly related to RukhsTy because the project aims to transform the driving license workflow from a mostly manual process into a structured digital service.

In Jordan, digital transformation is also a national priority. The Jordanian Digital Transformation Strategy 2026–2028 focuses on developing digital infrastructure, improving government digital services, strengthening digital trust, and using data and advanced technologies to improve service efficiency [3]. In addition, Jordan's Sanad platform is presented as an official digital gateway for accessing government services, digital documents, personal records, digital signatures, and bill payments [4]. The Driver and Vehicle Licensing Department also provides electronic portal services such as driver and vehicle information services and vehicle license renewal services [5]. However, these existing services do not fully represent a complete first-time driving license workflow that combines account registration, document upload, appointment booking, officer approvals, application tracking, payment simulation, digital license display, and delivery request in one academic system.

Several regional and international platforms show how driving license and transport services can be supported digitally. In Saudi Arabia, the Absher platform allows citizens and residents to access traffic services, including driving license-related services and appointment booking through the national platform [6]. Absher is a strong example of centralized government service delivery because it connects users with multiple government services through one digital channel. However, it is designed for the Saudi administrative and legal environment, so its workflow cannot be directly applied to Jordan without local adaptation.

In the United Arab Emirates, digital transport services provide online procedures for driving license services, including application submission, fee preview, payment, and service completion steps [7]. This shows the value of connecting transport services with online payment and user guidance. However, UAE systems depend on institutional integration and official production-level services, while RukhsTy focuses on a graduation project environment and uses safe mock integrations instead of real external APIs.

In the United Kingdom, the Driver and Vehicle Licensing Agency provides structured online services for driving license and vehicle-related transactions. The strength of this model is that it clearly guides users through service requirements and reduces the need for unnecessary office visits. However, the UK system is based on a

different legal, administrative, and identity-verification framework, which means it cannot be copied directly into the Jordanian context.

From a security perspective, research on online public services shows that moving government services online can improve access and efficiency, but it also increases the importance of secure communication, trusted authentication, and protection against vulnerabilities [8]. This supports the design decisions in RukhsTy, such as using role-based access control, password hashing, JWT authentication, and separate dashboards for different user roles.

Based on this comparison, existing platforms provide valuable ideas such as centralized access, digital identity, appointment booking, online payment, and service tracking. However, they either focus on a different country context, provide only part of the licensing process, or depend on official integrations that are outside the scope of a graduation project. RukhsTy fills this gap by presenting a Jordan-focused academic prototype that combines the main stages of the first-time driving license process into one organized workflow. The system supports citizens, admins, and officers, and demonstrates how application submission, document upload, appointment booking, officer review, result recording, mock payment, digital license issuing, and optional delivery can be managed through a single digital platform.

1.4 Problem Statement

The current driving license process in Jordan is still heavily dependent on manual and in-person procedures. Applicants may need to visit traffic offices several times to submit documents, check requirements, schedule tests, follow up on approvals, and complete the license issuing process. This causes long waiting times, repeated effort, and lack of clarity for citizens.

Manual document handling can also lead to several problems, such as missing files, duplicated data entry, delayed verification, and inconsistent records. Applicants may not always know the exact status of their application or the next required step. This creates uncertainty and forces citizens to depend on physical visits or direct communication to receive updates.

From the administrative side, officers and employees need to handle many applications manually. This increases workload, makes tracking harder, and may delay decisions. Different officers are responsible for different stages, such as security review, medical test, theory exam, and practical exam. Without a unified system, coordination between these stages can become slow and unclear.

Another limitation is that payment, license issuing, QR verification, and delivery are not always presented to the citizen as one smooth digital process. A citizen may complete approval but still need to follow separate steps to pay, receive, or verify the license. Therefore, there is a need for a unified digital platform that organizes the complete licensing workflow from application submission to final license issuing and delivery.

Rukhsty addresses this problem by providing a role-based web application that supports citizens, admins, and officers through a structured workflow. The system improves transparency, reduces paperwork, supports status tracking, and demonstrates payment and delivery flows through safe mock integrations.

1.5 Project Objectives

The main objective of RukhsTy is to design and develop a digital driving license management system that simplifies and organizes the process of applying for, reviewing, approving, issuing, paying for, and delivering driving licenses.

The specific objectives of the project are:

1. To develop a secure registration and login module that allows citizens, admins, and officers to access the system according to their assigned roles.
2. To enable citizens to submit driving license applications online, upload required documents, and complete the required application information through a structured digital form.
3. To provide an application tracking feature that allows citizens to view the current status and progress of their submitted applications throughout the licensing workflow.
4. To implement role-based dashboards for Admin, Security Officer, Medical Officer, Theory Officer, and Practical Officer to manage their responsibilities in the license approval process.
5. To allow authorized officers to review applications, approve or reject requests, record medical test results, record theory exam results, and record practical exam results.
6. To provide digital service completion features, including digital license display, QR code verification, mock eFAWATEERcom payment, and mock Aramex delivery request for demonstration purposes.
7. To apply secure and organized system design using JWT authentication, password hashing, role-based access control, and a structured database to protect user data and support reliable system operation.

1.6 Project Plan

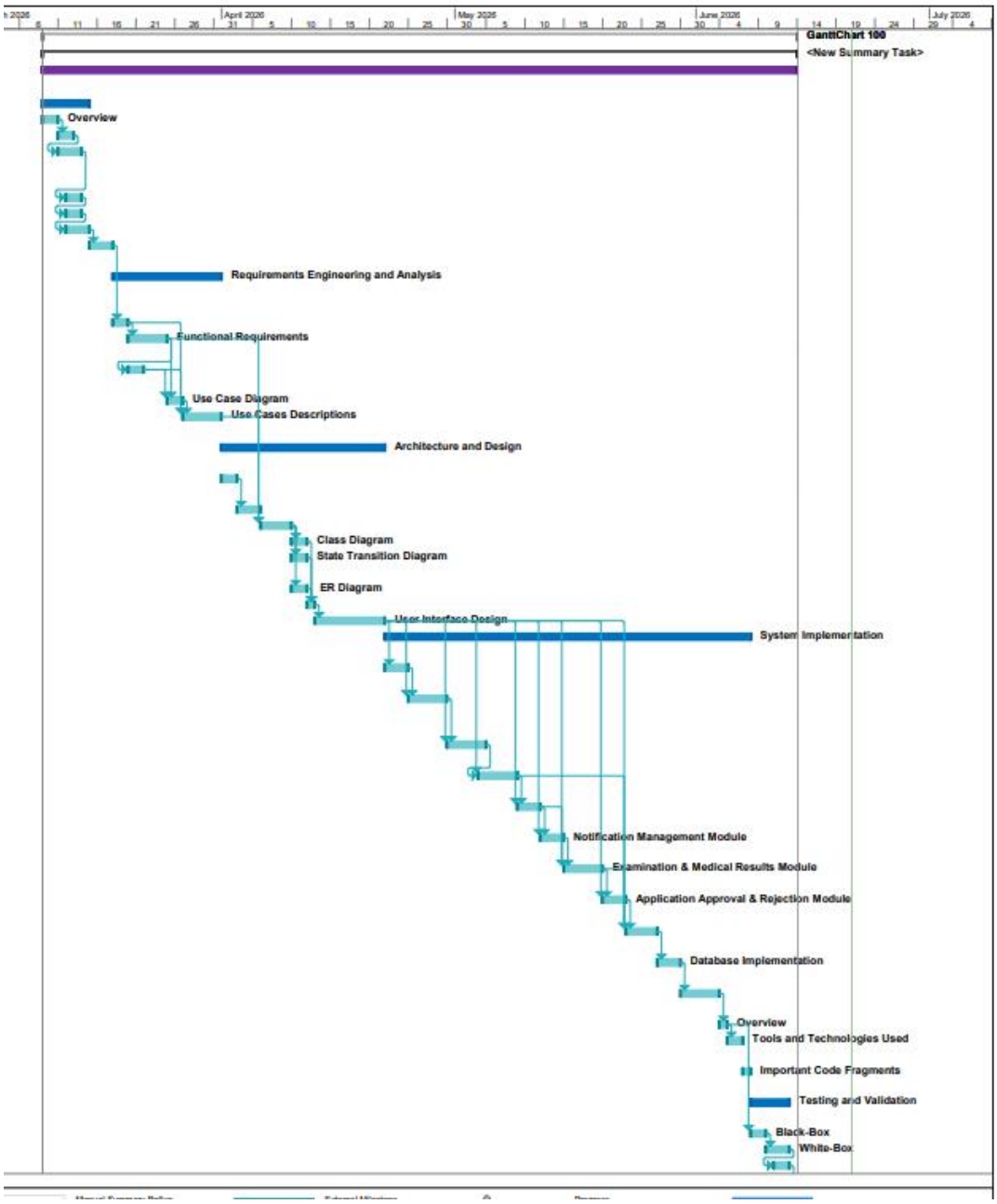
The RukhsTy project was planned and developed using an Agile iterative approach. This approach was selected because the system includes several connected workflows, different user roles, and features that required continuous testing, feedback, and improvement. Instead of completing the project in one fixed sequence, the work was divided into short iterations. Each iteration focused on designing, implementing, testing, and improving a specific part of the system.

The project plan started with requirements gathering and analysis, where the team studied the driving license process, identified the main stakeholders, and defined the functional and non-functional requirements. After that, the team prepared the main system diagrams, including the use case diagram, use case descriptions, sequence diagrams, class diagram, state transition diagram, ER diagram, and activity diagram.

The next iterations focused on implementation. The frontend was developed using React, TypeScript, and Tailwind CSS, while the backend was implemented using Node.js, Express.js, and PostgreSQL. The main modules were developed gradually, including authentication, user registration, application submission, appointment booking, officer dashboards, medical and exam result management, application approval and rejection, payment simulation, digital license issuing, and delivery request.

Testing and validation were performed throughout the development process, not only at the end of the project. Each completed module was tested after implementation to ensure that it worked correctly and did not break previous features. The final iteration focused on system integration, user interface improvements, documentation, screenshots, testing tables, and final report preparation.

The Gantt chart in Figure X shows the planned schedule of the project activities, including requirements engineering and analysis, software design, user interface design, system implementation, database implementation, testing and validation, and documentation. The chart demonstrates how the Agile iterations were organized across the project timeline.



The Gantt chart shows the project schedule and the main activities completed during the development of the RukhsTy system. It includes requirements engineering and analysis, functional requirements, use case diagram and descriptions, architecture and design, sequence diagrams, class diagram, state transition diagram, ER diagram, user interface design, system implementation, database implementation, and testing and validation. The chart supports the Agile development approach by showing how project tasks were completed through organized iterations and continuous progress across the project timeline.

Chapter 2: The Development Process

RukhsTy was developed using an Agile iterative development process. This approach was selected because the system includes several connected workflows, multiple user roles, and features that required continuous testing, feedback, and improvement. Agile development allowed the team to build the system gradually, test each completed part, and improve the platform based on results and feedback instead of developing all features at once.

The project was divided into short development iterations. Each iteration focused on a specific group of related tasks and produced a working part of the system. This helped the team manage the project more clearly, identify issues early, and make improvements throughout the development process.

The first iteration focused on project setup, requirements understanding, database planning, and authentication. During this stage, the team prepared the development environment, organized the project structure, designed the initial database schema, and implemented registration and login with role-based access control.

The second iteration focused on the citizen workflow. This included the citizen dashboard, driving license application submission, document upload, appointment booking, and application tracking. The purpose of this iteration was to allow citizens to start and follow their license application digitally.

The third iteration focused on officer workflows. The Security Officer dashboard was developed to support application review, document checking, approval, and rejection. After that, the Medical Officer, Theory Officer, and Practical Officer workflows were implemented to record test results and update the application stage.

The fourth iteration focused on service completion features. This included digital license issuing, QR code verification, mock eFAWATEERcom payment, and mock Aramex delivery request. These features were added to demonstrate a complete end-to-end driving license process in a safe academic environment without using real external integrations.

The final iteration focused on integration, testing, bug fixing, user interface improvements, documentation, and final project preparation. The system was tested

using different user roles to ensure that each role could access only the correct features and that the application workflow worked as expected.

Throughout the development process, GitHub was used for version control and team collaboration. The team also prepared diagrams, screenshots, testing tables, and documentation to support the final graduation report. This Agile iterative process helped ensure that RukhsTy was developed in an organized, testable, and maintainable way.

Chapter 3: Requirements Engineering and Analysis

3.1 Stakeholders

The Rukhsty system includes several stakeholders who interact with the platform or are affected by its results. Each stakeholder has different needs and responsibilities within the driving license workflow.

1. **Citizen / User**

The citizen is the main user of the system. Citizens use Rukhsty to register, log in, submit driving license applications, upload required documents, track application status, book appointments, pay the license fee through the mock eFAWATEERcom flow, view the issued digital license, verify the QR code, and request delivery.

2. **Admin**

The Admin is responsible for managing the system and monitoring operations. The Admin can manage users, officers, applications, reports, services, and license-related data. This role helps ensure that the platform operates correctly and that system information remains organized.

3. **Security Officer**

The Security Officer is responsible for reviewing submitted applications. This includes checking applicant information, viewing uploaded documents, and deciding whether to approve or reject the application. The security review stage is important because it determines whether the application can continue to the next stages.

4. **Medical Officer**

The Medical Officer is responsible for managing the medical stage of the application. This role records medical test results and updates the application status based on whether the applicant passes or fails the required medical check.

5. **Theory Officer**

The Theory Officer is responsible for managing the theory exam stage. This role records theory exam results and updates the application status. If the applicant passes the theory exam, the application can continue to the practical exam stage.

6. **Practical Officer**

The Practical Officer is responsible for managing the practical driving exam stage. This role records practical exam results and updates the application status. If the

applicant passes the practical exam, the application can move toward payment and license issuing.

7. System Developers and Maintainers

System developers and maintainers are responsible for developing, testing, updating, and maintaining the Rukhsty platform. They ensure that the system remains secure, functional, and ready for future improvements.

8. University Supervisor and Evaluators

The university supervisor and evaluators are stakeholders in the academic context of the project. They review the project, provide feedback, evaluate the technical work, and assess whether the system meets graduation project requirements.

3.2 Functional Requirements

ID	Requirement	Description	Actor	Priority
FR-01	User Registration	The system shall allow citizens to create a new account using an email address and password. The system shall verify that the email address is unique before creating the account.	Citizen	High
FR-02	User Login / Logout	The system shall allow all users to log in using an email address and password. The system shall authenticate users securely and allow them to log out of the system.	All Roles	High
FR-03	Apply for Driving License	The system shall allow citizens to submit a new driving license application by selecting the required license service.	Citizen	High
FR-04	Upload Documents	The system shall allow citizens to upload the required documents, including a personal photo. The system shall validate that all required information and documents are provided before submission.	Citizen	High
FR-05	Track Application Status	The system shall allow citizens to view their applications, track the current application status, and monitor the progress of each application through its processing stages.	Citizen	High
FR-06	Book Appointment	The system shall allow citizens to book appointments for different licensing stages, including training, medical examination, theory examination, and practical examination. The system shall record the appointment date, center, and appointment type.	Citizen	High
FR-07	Receive Notifications	The system shall send in-app notifications to citizens regarding application status updates, appointment confirmations, and examination results.	Citizen	Medium
FR-08	Record Exam / Medical Results	The system shall allow authorized officers to record pass or fail results for medical examinations, theory examinations, and practical examinations. The system shall update the application status based on the recorded results.	Officer	High
FR-09	Approve / Reject Applications	The system shall allow administrators to approve or reject driving license applications and provide a rejection reason when applicable. The system shall notify citizens of the application decision.	Admin	High
FR-10	Bilingual Interface (AR / EN)	The system shall provide a bilingual user interface in Arabic and English. The system shall support right-to-left (RTL) layout for Arabic content.	All Roles	Medium

Table 2

3.3 Non-Functional Requirements

ID	Requirement	**Priority**	**ID**	**Requirement**	**Priority**
NFR01	**Performance:** Key pages shall load in under 3 seconds under normal network conditions.	**High**	**NFR02**	**Accessibility:** The system shall support both Arabic and English languages.	**High**
NFR03	**Security:** The system shall secure user authentication using JWT tokens with expiration and revocation mechanisms, and all passwords shall be salted and hashed using bcrypt.	**High**	**NFR04**	**Availability:** The system shall achieve at least 99.5% uptime (excluding planned maintenance).	**Medium**
NFR05	**Scalability:** The system shall support at least 1000 concurrent users with acceptable response times.	**Medium**	**NFR06**	**Usability:** A new user shall be able to complete registration and submit an application within 10 minutes.	**High**
NFR07	**Maintainability:** The system shall follow a modular architecture with well-documented code.	**High**	**NFR8**	**Data integrity:** The system shall ensure atomic database operations and maintain audit logs for critical actions.	**High**

Table 3

3.4 Use Case Diagram

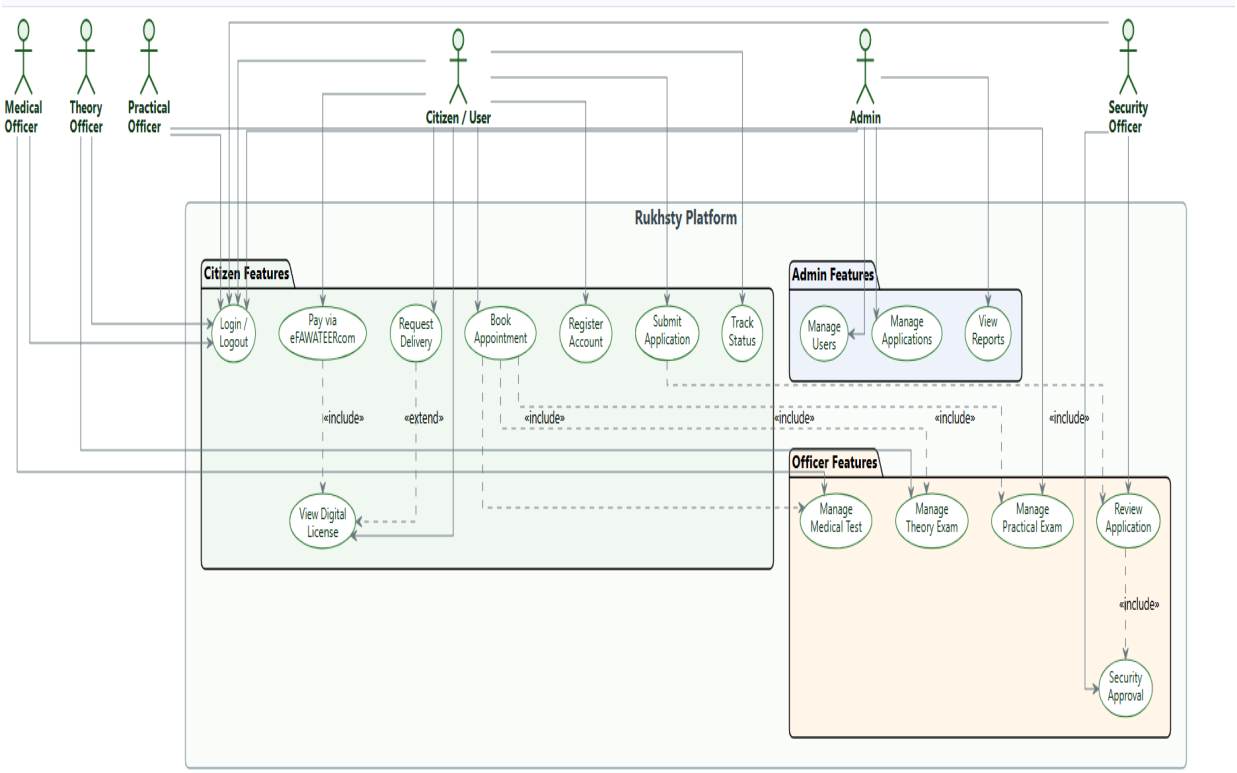


Figure 1. Use Case Diagram

The use case diagram illustrates the main interactions between the RukhsTy platform and its different actors, including the Citizen/User, Admin, Security Officer, Medical Officer, Theory Officer, and Practical Officer. The Citizen/User can register an account, log in, apply for eFAWATEERcom payment, request delivery, book appointments, submit applications, and track application status. The Admin is responsible for managing users, managing applications, and viewing reports. The officer roles handle the application review process by managing medical tests, theory exams, practical exams, reviewing applications, and approving security checks. This diagram shows how the platform organizes the driving-license process into clear citizen, admin, and officer functionalities.

3.5 Use Case Descriptions

This section describes the main use cases of the Rukhsty platform. Each use case includes the actor, brief description, preconditions, basic flow, alternative flow, and postconditions.

Use Case 1: Register Account

Field	Description
Use Case Name	Register Account
Main Actor	Citizen / User
Description	Allows a new citizen to create an account in the Rukhsty platform.
Precondition	The citizen does not already have an account.
Basic Flow	1. The citizen opens the registration page. 2. The citizen enters personal information such as name, national ID, email, password, and profile details. 3. The system validates the entered data. 4. The system creates a new user account. 5. The citizen can log in to the system.
Alternative Flow	If the email or national ID already exists, the system shows an error message.
Postcondition	A new citizen account is created successfully.

Use Case 2: Login / Logout

Field	Description
Use Case Name	Login / Logout
Main Actor	Citizen / User, Admin, Security Officer, Medical Officer, Theory Officer, Practical Officer
Description	Allows all system users to access their dashboards securely according to their role.
Precondition	The user must have a registered account.
Basic Flow	1.The user opens the login page. 2.The user enters email or national ID and password. 3.The system validates the credentials. 4.The system redirects the user to the correct dashboard based on role. 5. The user can log out when finished.
Alternative Flow	If the credentials are invalid, the system displays an error message.
Postcondition	The user is authenticated and redirected to the correct dashboard.

Use Case 3: Submit Application

Field	Description
Use Case Name	Submit Application
Main Actor	Citizen / User
Description	Allows the citizen to submit a driving license application through the platform.
Precondition	The citizen must be logged in.
Basic Flow	1. The citizen opens the driving license service page. 2. The citizen selects the required license service. 3. The citizen fills in the application details. 4. The citizen uploads the required documents or training certificate. 5. The system validates the application data. 6. The system submits the application for review.
Alternative Flow	If required information or documents are missing, the system prevents submission and shows a validation message.
Postcondition	The application is submitted and sent to the security review stage.

Use Case 4: Track Status

Field	Description
Use Case Name	Track Status
Main Actor	Citizen / User
Description	Allows the citizen to track the current status and progress of the submitted application.
Precondition	The citizen must have at least one submitted application.
Basic Flow	1. The citizen opens the application tracking page. 2. The system displays all submitted applications. 3. The citizen selects an application. 4. The system displays the current status and workflow progress.
Alternative Flow	If the citizen has no applications, the system displays an empty application message.
Postcondition	The citizen views the latest application status.

Use Case 5: Book Appointment

Field	Description
Use Case Name	Book Appointment
Main Actor	Citizen / User
Description	Allows the citizen to book appointments for medical test, theory exam, or practical exam.
Precondition	The application must reach a stage that requires appointment booking.

Field	Description
Basic Flow	1. The citizen opens the appointment booking page. 2. The citizen selects the appointment type. 3. The citizen chooses an available date and time. 4. The system confirms the appointment.
Alternative Flow	If no appointment slots are available, the system asks the citizen to choose another date.
Postcondition	The appointment is booked and linked to the application.

Use Case 6: Security Approval

Field	Description
Use Case Name	Security Approval
Main Actor	Security Officer
Description	Allows the security officer to approve or reject the application after review.
Precondition	The application must be reviewed by the security officer.
Basic Flow	1. The officer opens the reviewed application. 2. The officer checks the applicant information and documents. 3. The officer chooses approve or reject. 4. The system updates the application status.
Alternative Flow	If the application is rejected, the system records the rejection reason and stops the workflow.
Postcondition	The application moves to the next stage if approved, or becomes rejected if not approved.

Use Case 7: Manage Medical Test

Field	Description
Use Case Name	Manage Medical Test
Main Actor	Medical Officer
Description	Allows the medical officer to manage medical test results for license applicants.
Precondition	The medical officer must be logged in and the application must reach the medical stage.
Basic Flow	1. The medical officer opens the medical test dashboard. 2. The system displays applicants waiting for medical test results. 3. The officer selects an applicant. 4. The officer records the medical test result. 5. The system updates the application status.

Field	Description
Alternative Flow	If the applicant fails the medical test, the system marks the stage as failed or rejected.
Postcondition	The medical test result is saved.

Use Case 8: Manage Theory Exam

Field	Description
Use Case Name	Manage Theory Exam
Main Actor	Theory Officer
Description	Allows the theory officer to manage and record theory exam results.
Precondition	The theory officer must be logged in and the application must reach the theory exam stage.
Basic Flow	1. The theory officer opens the theory exam dashboard. 2. The system displays applicants waiting for theory exam results. 3. The officer selects an applicant. 4. The officer records the theory exam result. 5. The system updates the application status.
Alternative Flow	If the applicant fails the theory exam, the system keeps the application in the theory stage or marks it as failed.
Postcondition	The theory exam result is saved.

Use Case 9: Manage Practical Exam

Field	Description
Use Case Name	Manage Practical Exam
Main Actor	Practical Officer
Description	Allows the practical officer to manage and record practical driving exam results.
Precondition	The practical officer must be logged in and the application must reach the practical exam stage.
Basic Flow	1. The practical officer opens the practical exam dashboard. 2. The system displays applicants waiting for practical exam results. 3. The officer selects an applicant. 4. The officer records the practical exam result. 5. The system updates the application status.
Alternative Flow	If the applicant fails the practical exam, the system keeps the application in the practical stage or marks it as failed.

Field	Description
Postcondition	The practical exam result is saved and the application can move to license issuing if passed.

Use Case 10: Pay by eFAWATEERcom

Field	Description
Use Case Name	Pay by eFAWATEERcom
Main Actor	Citizen / User
Description	Allows the citizen to pay the license fee using a mock eFAWATEERcom payment flow for demo purposes.
Precondition	The license application must be approved and ready for payment.
Basic Flow	1. The citizen opens the approved application page. 2. The system displays payment details and amount. 3. The citizen clicks “Pay by eFAWATEERcom”. 4. The system simulates successful payment. 5. The payment status changes to paid.
Alternative Flow	If the payment process fails, the system keeps the payment status as unpaid and displays an error message.
Postcondition	The payment is marked as paid and the digital license becomes available.

Chapter 4: Architecture and Design

4.1 Software Architecture

The RukhsTy system follows a Layered Architecture, also known as a Three-Tier Architecture. This architecture was selected because it separates the system into clear layers: the presentation layer, the business logic layer, and the data layer. This separation makes the system easier to develop, test, maintain, and update.

The architecture of RukhsTy consists of the following main layers:

1. Presentation Layer

The presentation layer represents the frontend user interface of the system. It is used by citizens, administrators, security officers, medical officers, theory officers, and practical officers. This layer allows users to interact with the system through pages such as registration, login, application submission, appointment booking, application tracking, officer dashboards, and digital license viewing. The presentation layer also handles basic input validation and sends user requests to the backend API.

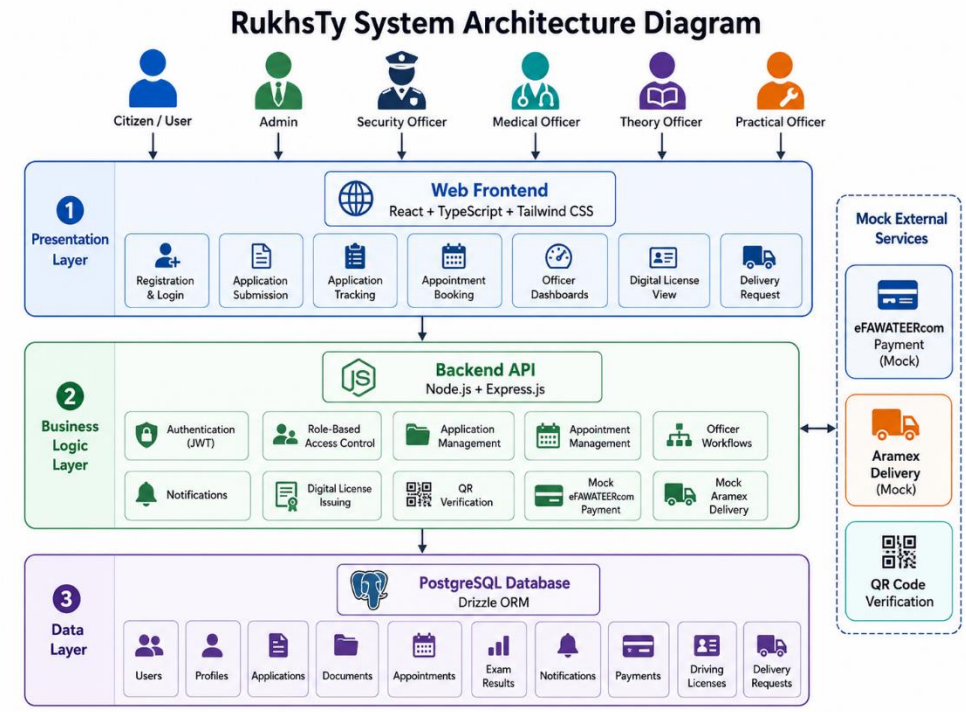
2. Business Logic Layer

The business logic layer represents the backend part of the system. It is responsible for processing user requests and applying the main system rules. This layer manages authentication, role-based access control, application submission, appointment handling, officer workflows, medical and exam result recording, application approval and rejection, notifications, mock payment processing, and digital license issuing. It also controls the communication between the frontend and the database.

3. Data Layer

The data layer represents the database and data management part of the system. It stores and manages system data such as users, profiles, applications, uploaded documents, appointments, notifications, medical results, theory exam results, practical exam results, payments, licenses, and delivery requests. This layer helps maintain data consistency, integrity, and reliability.

The system architecture also includes security mechanisms such as JWT authentication, password hashing, and role-based access control. These mechanisms ensure that each user can only access the features related to their assigned role. In addition, the system includes safe mock integrations for eFAWATEERcom payment and Aramex delivery to demonstrate the complete licensing workflow without using real external APIs.



The system architecture diagram illustrates the main structure of the RukhsTy platform. It shows how citizens, admins, and officers interact with the frontend interface, how requests are sent to the backend API, and how the backend processes business logic such as authentication, application management, appointment booking, officer workflows, payment simulation, and license issuing. The diagram also shows how the backend communicates with the PostgreSQL database to store and retrieve system data. This architecture separates the user interface, business logic, and data management layers to improve maintainability, security, and system organization.

4.2 Software Design

4.2.1 Sequence Diagrams

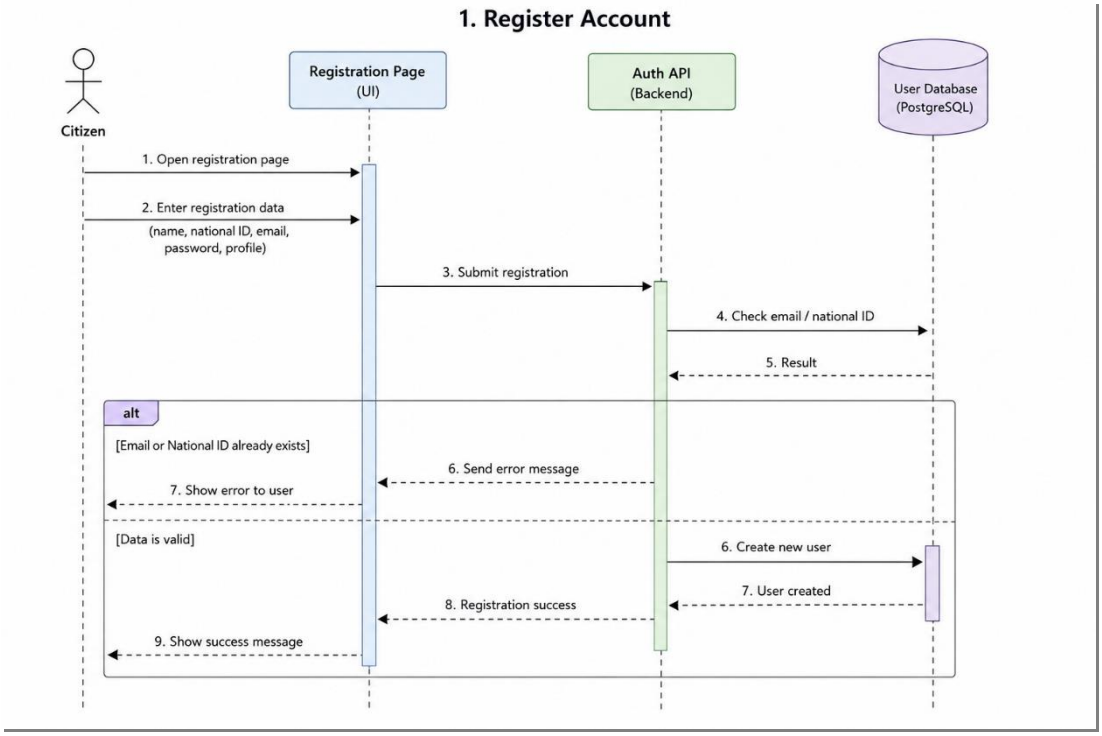


Figure 2. Register Account Sequence Diagram

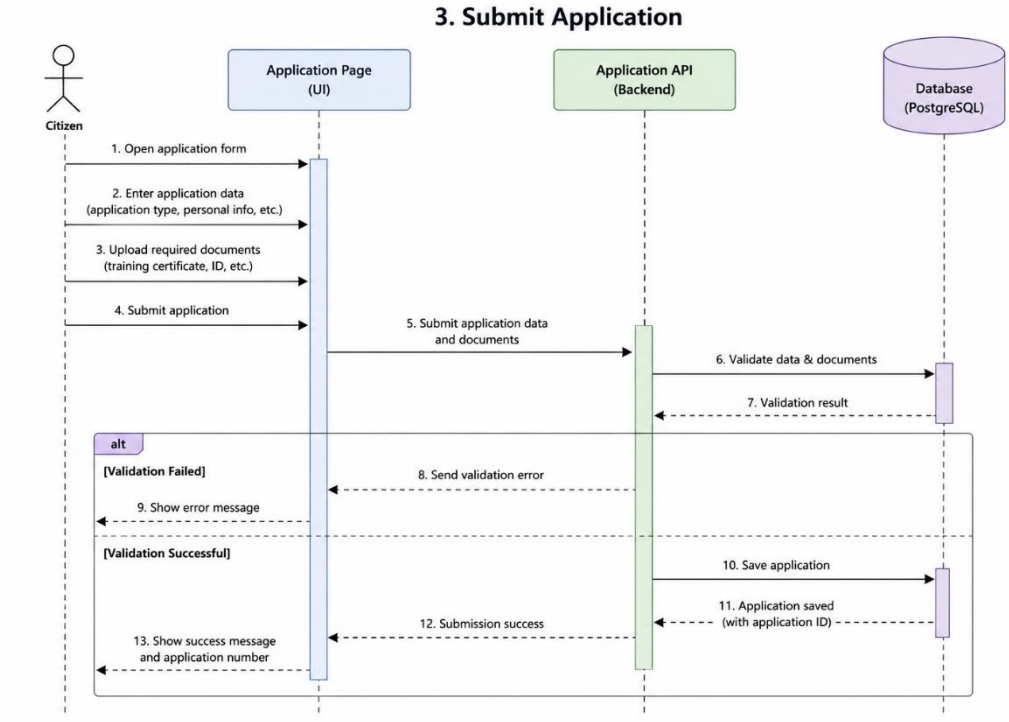


Figure 3. Submit Application Sequence Diagram

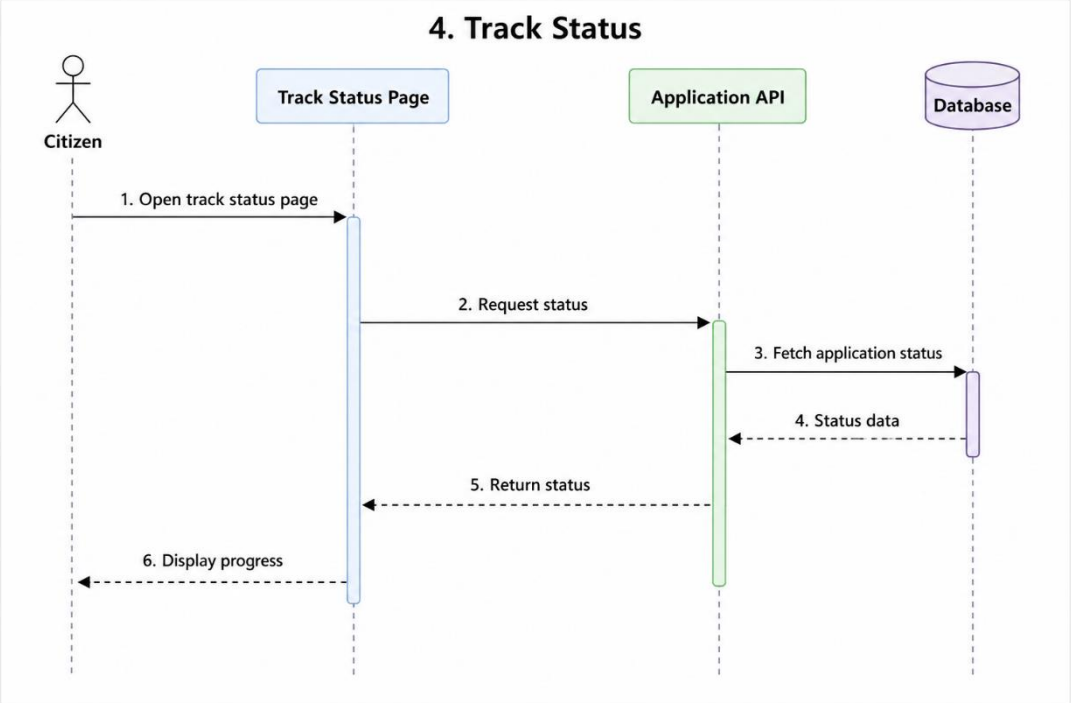


Figure 4. Track Status Sequence Diagram

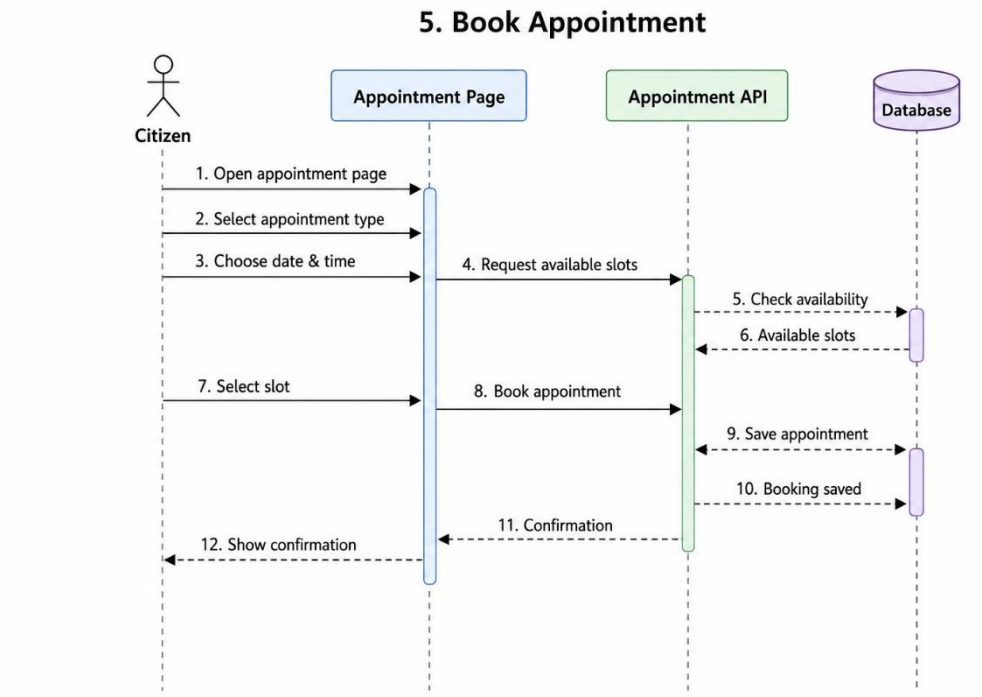


Figure 5. Book Appointment Sequence Diagram

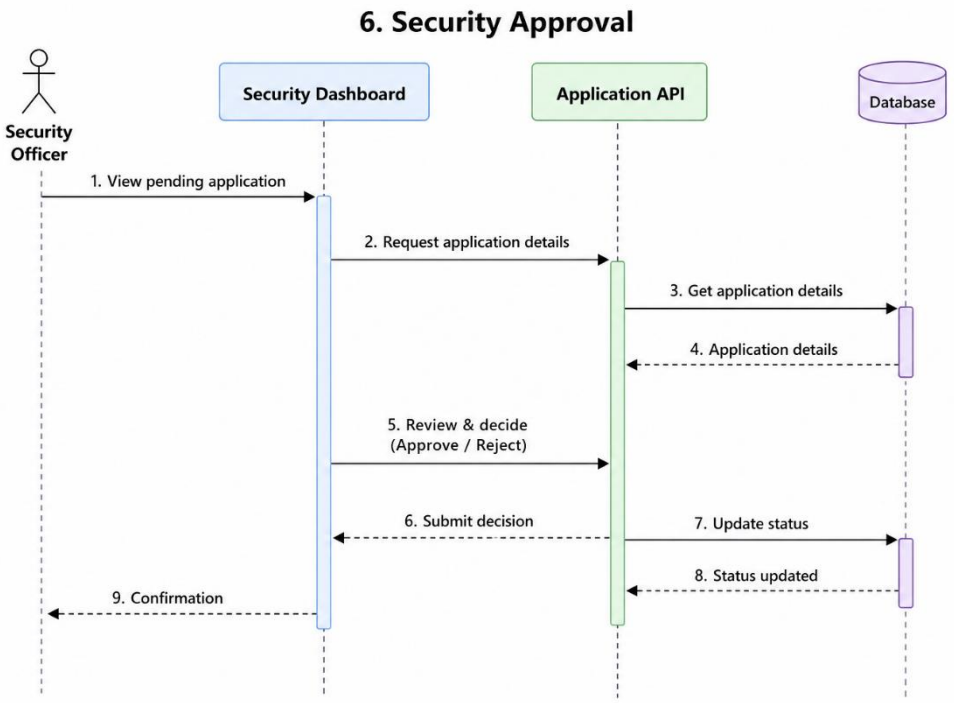


Figure 6. Security Approval Sequence Diagram

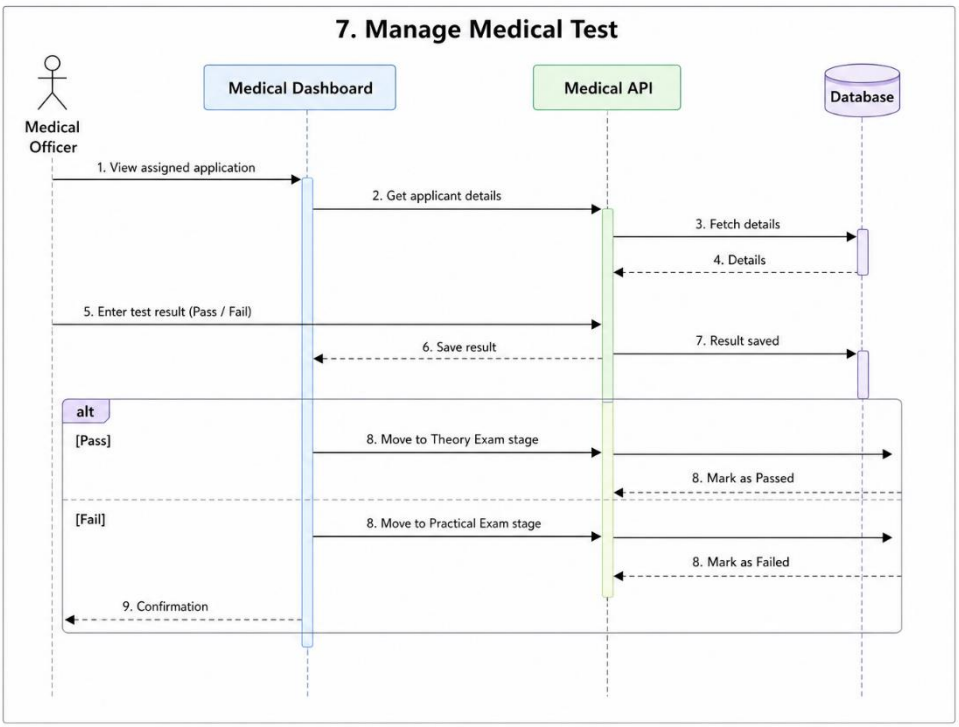


Figure 7. Manage Medical Test Sequence Diagram

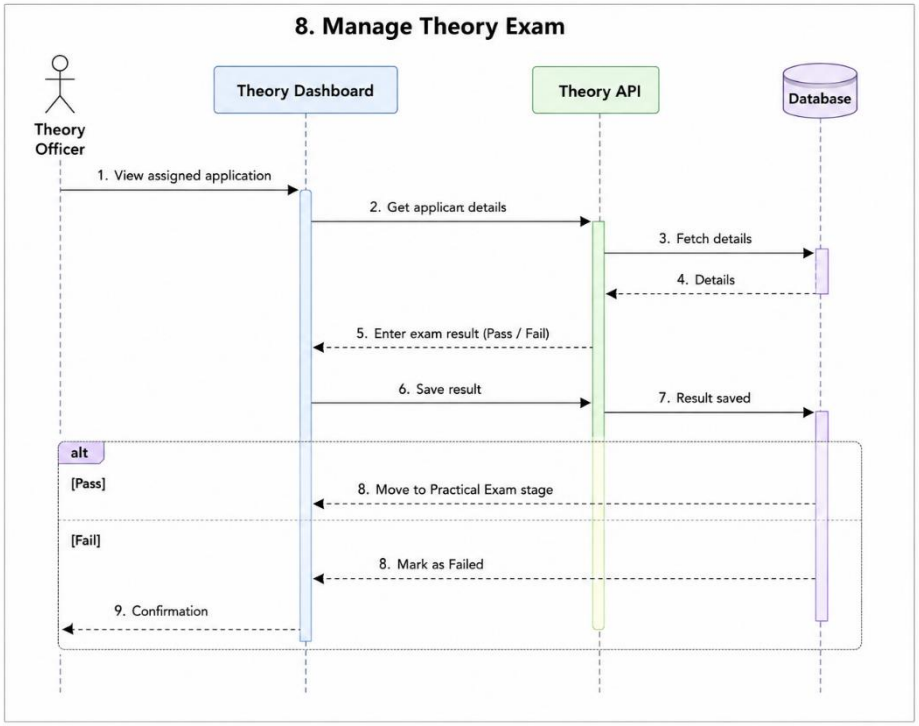


Figure 8. Manage Theory Exam Sequence Diagram

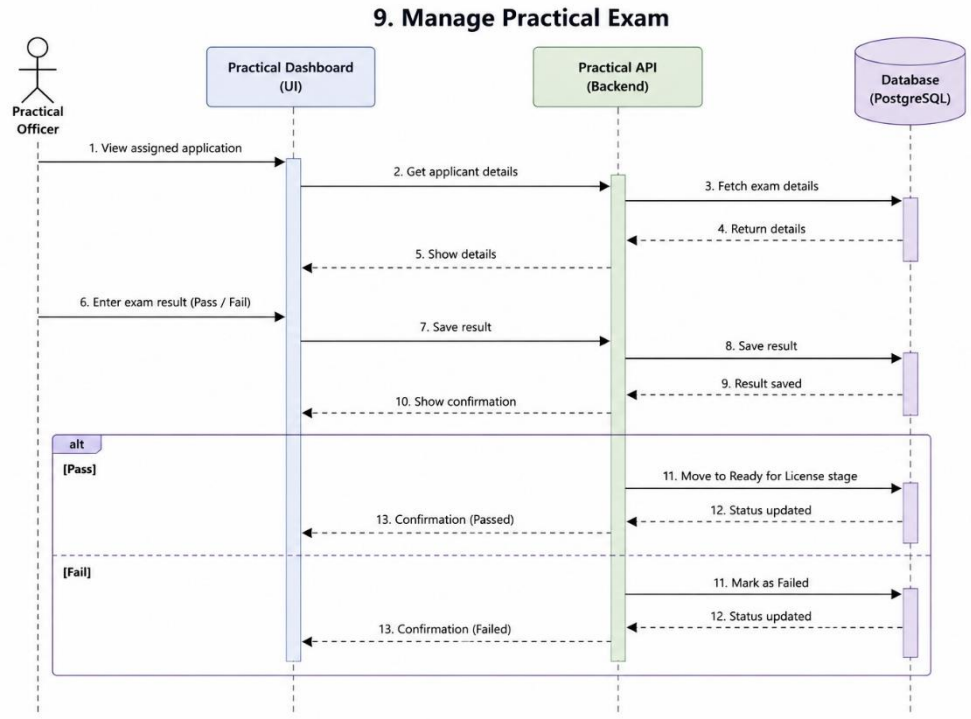


Figure 9. Manage Practical Exam Sequence Diagram

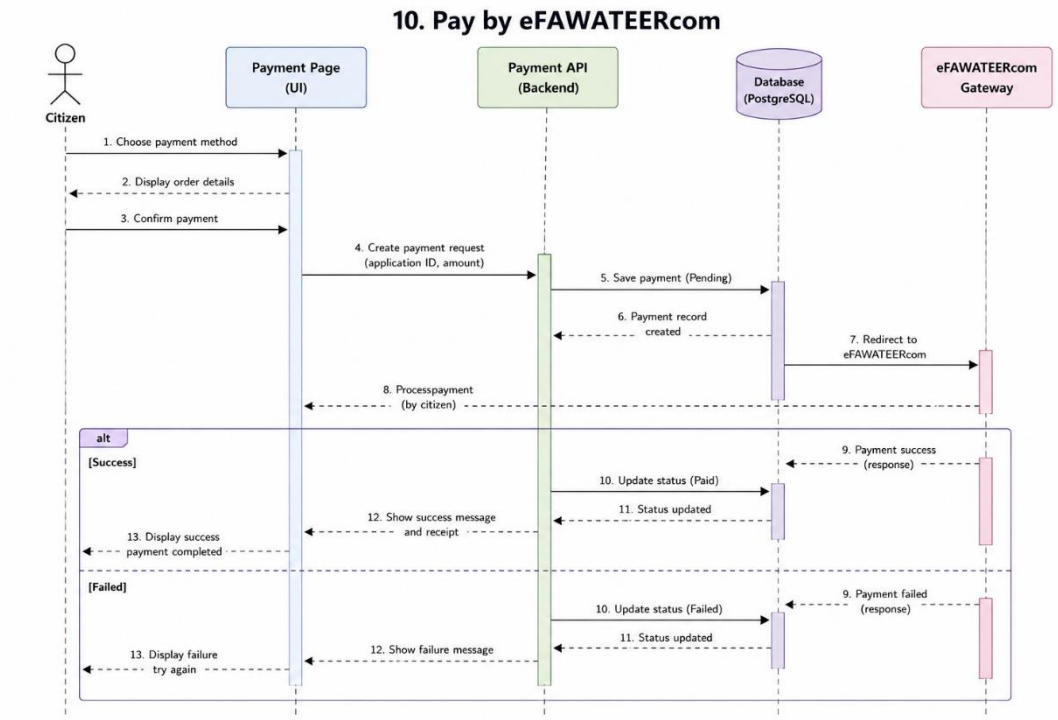


Figure 10. Pay by eFAWATEERcom Sequence Diagram

4.2.2 Class Diagram

In software engineering, a class diagram is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

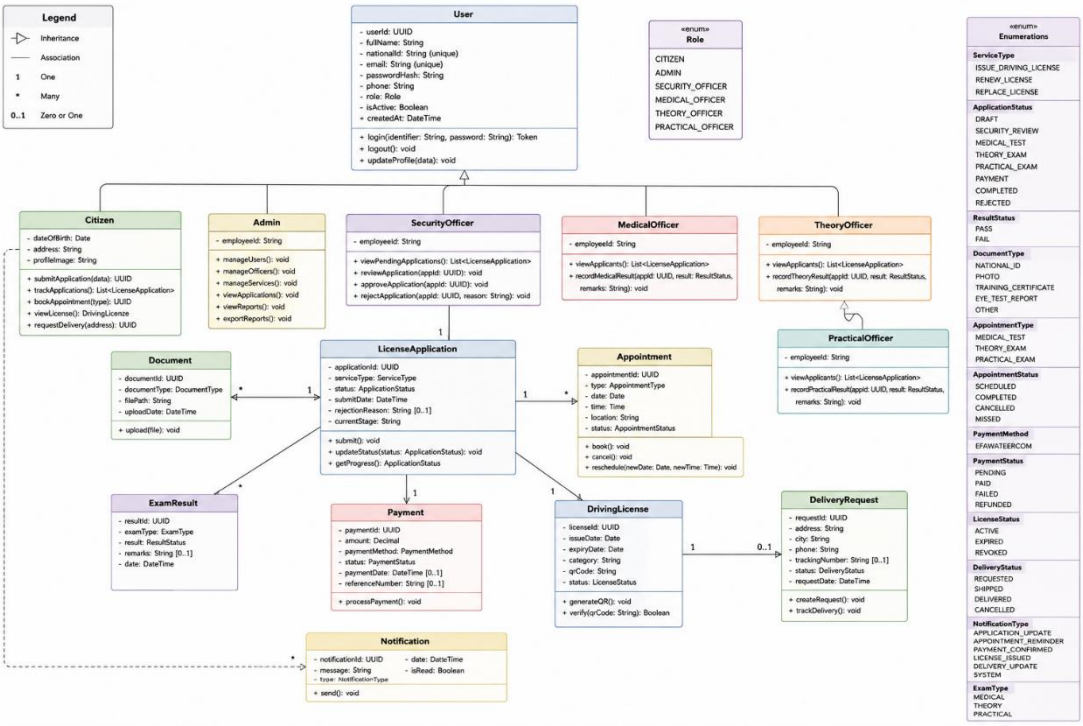


Figure 11. Class Diagram

4.2.3 State Transition Diagram

Provide state transition diagram of the system and if needed for some components/classes.

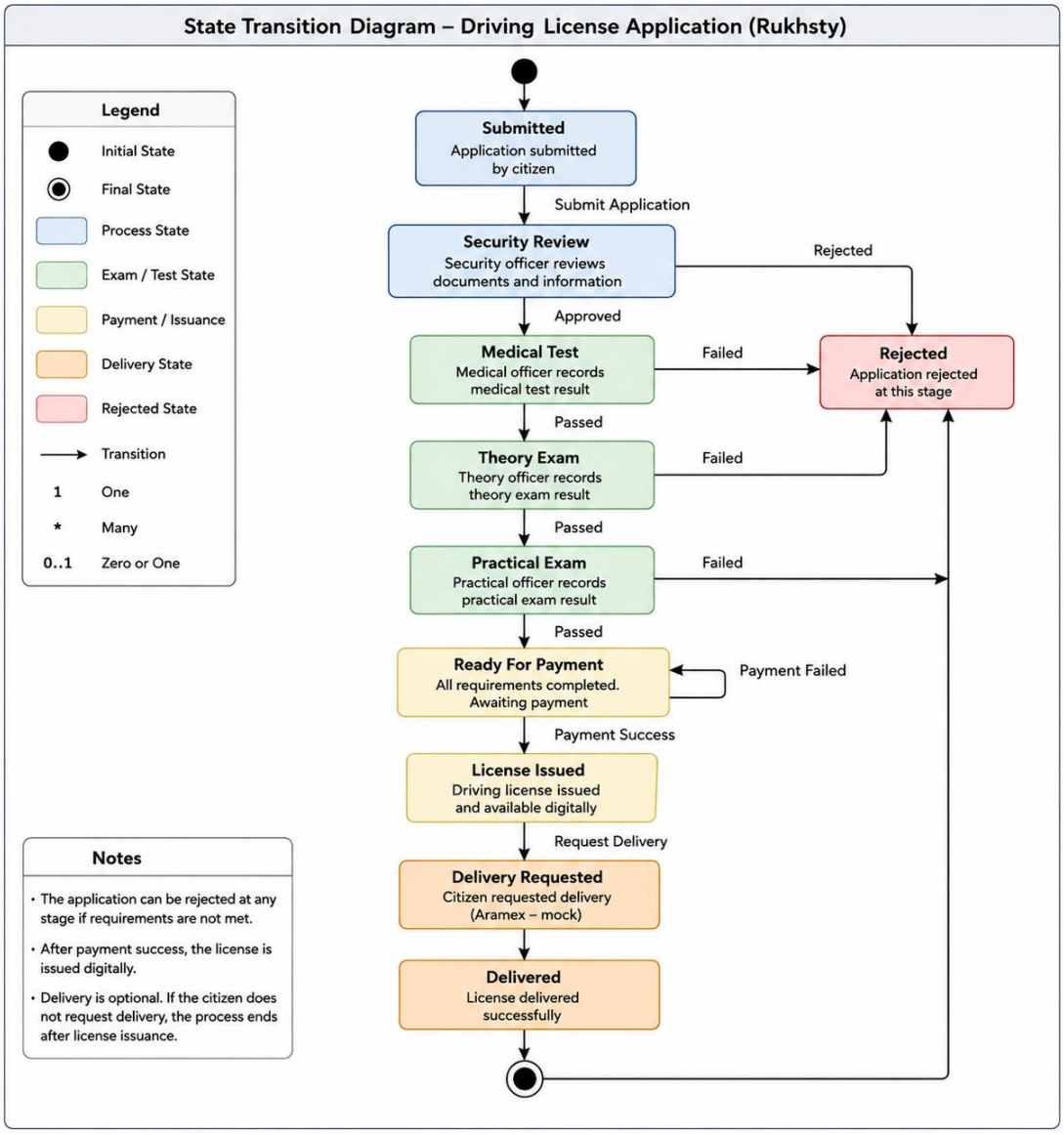


Figure 12. State Transition Diagram

4.2.4 ER Diagram

Entity relationship diagram displays the relationships of entity set stored in a database.

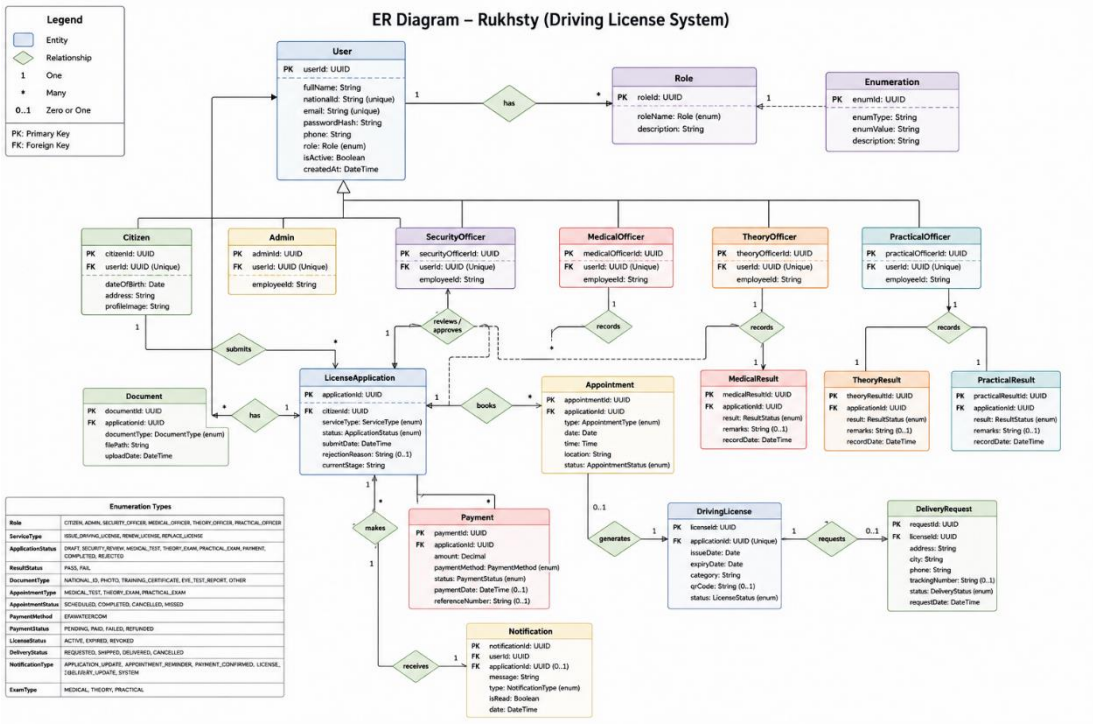


Figure 13. ER Diagram

4.2.5 Activity Diagram

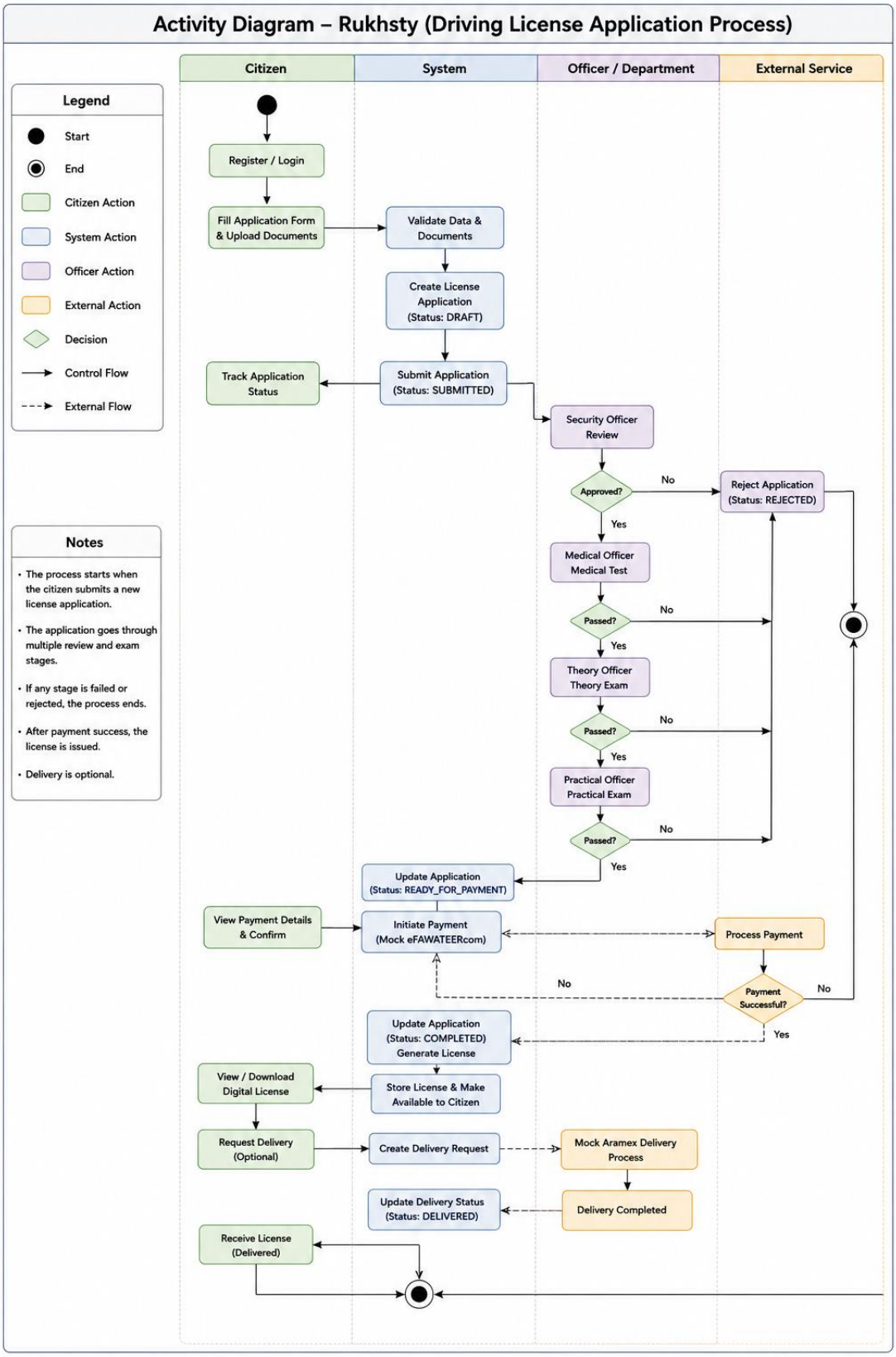


Figure 14. Activity Diagram

4.3 User Interface Design

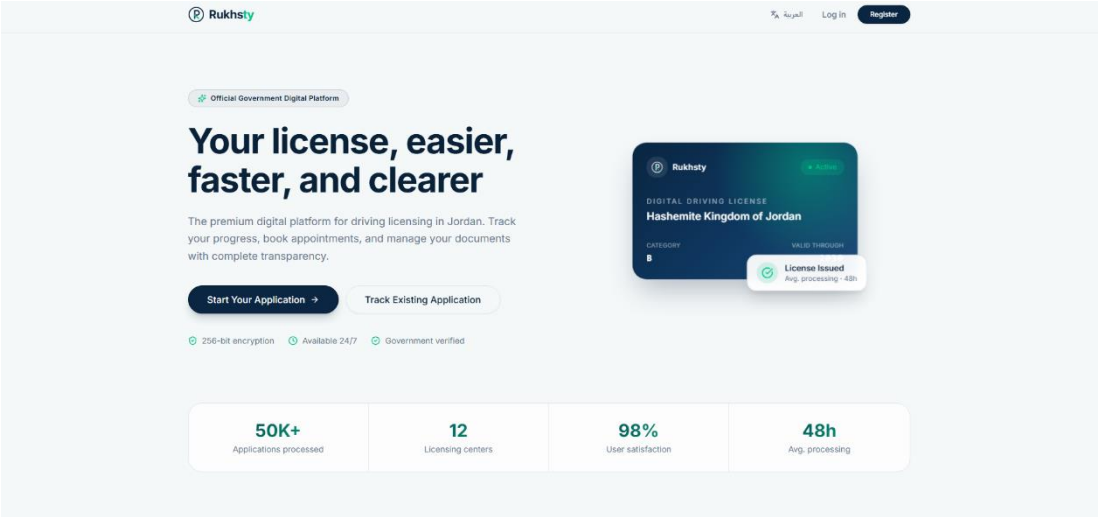


Figure 15. RukhsTy Home Page

Rukhsty Home Page

This page shows the first screen users see when they open Rukhsty. From here, users can start a new license application or track an existing one.

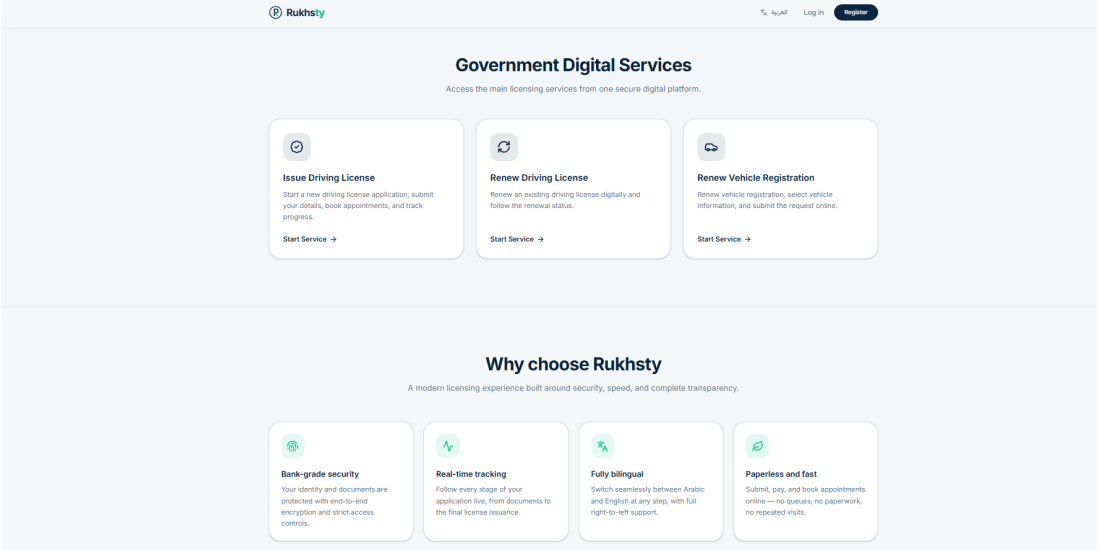


Figure 16. RukhsTy Services Page

Rukhsty Services Page

This page shows the main services available in Rukhsty, including issuing a driving license, renewing a driving license, and renewing vehicle registration.

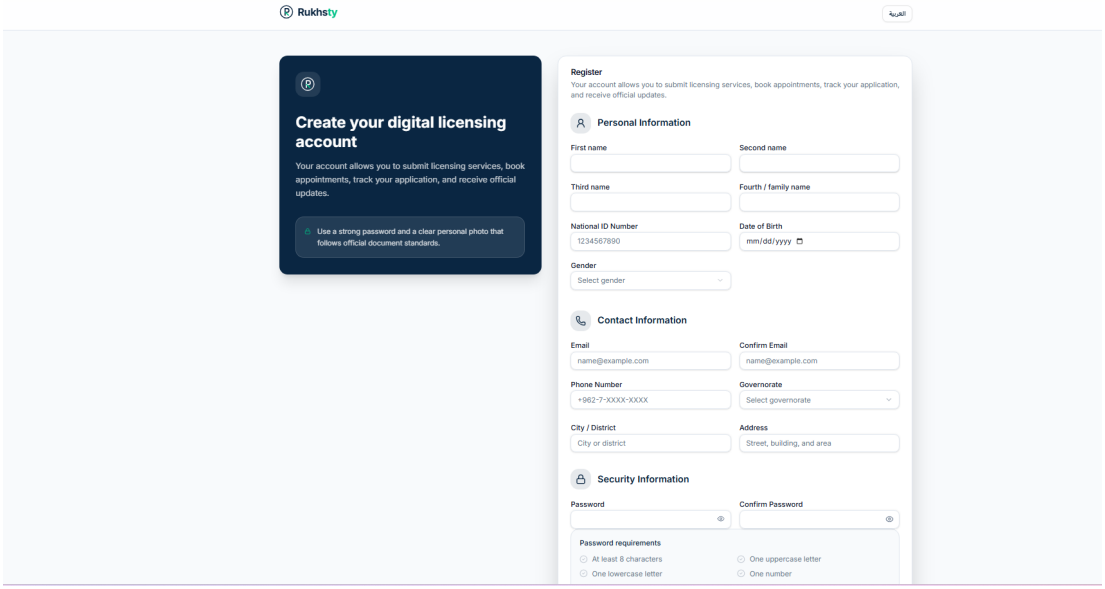


Figure 17. Register Page

Register Page

This page allows a new user to create an account by entering personal, contact, and security information**.**

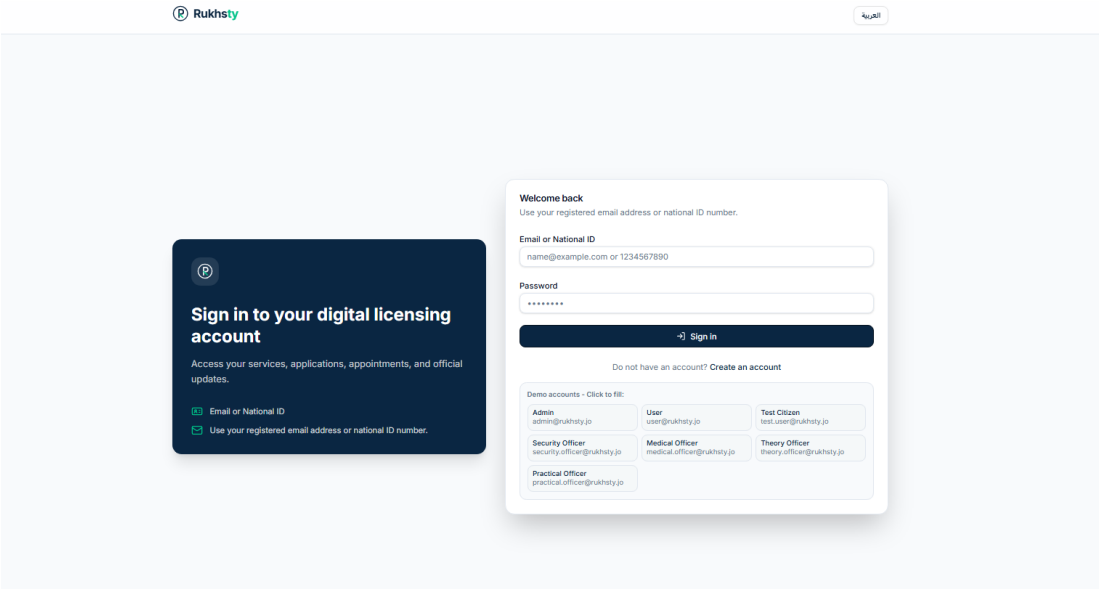


Figure 18. Login Page

Login Page

This page allows users and officers to sign in using their email or national ID with a password**.**

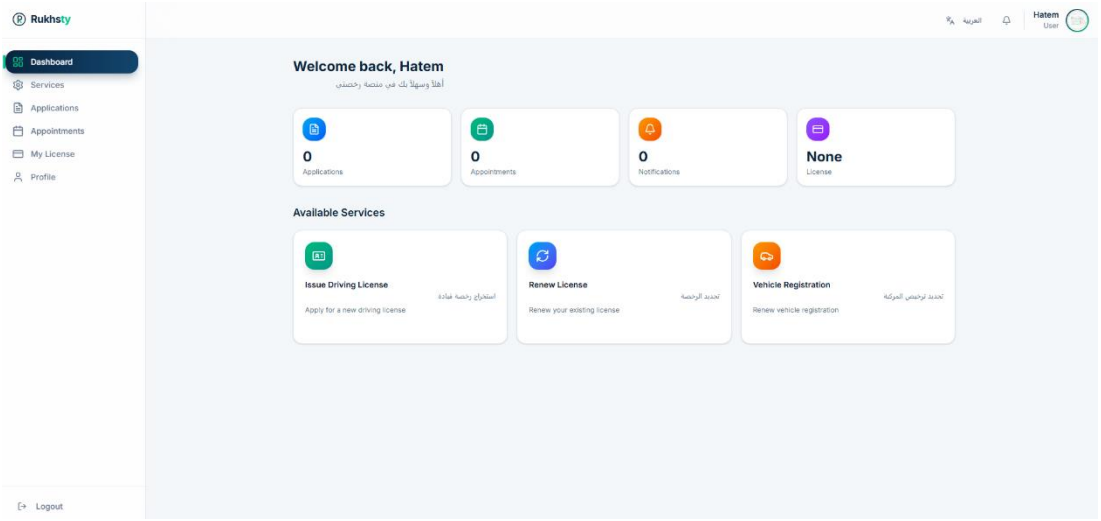


Figure 19. User Dashboard

User Dashboard

This page shows the user's main dashboard with applications, appointments, notifications, license status, and available services.

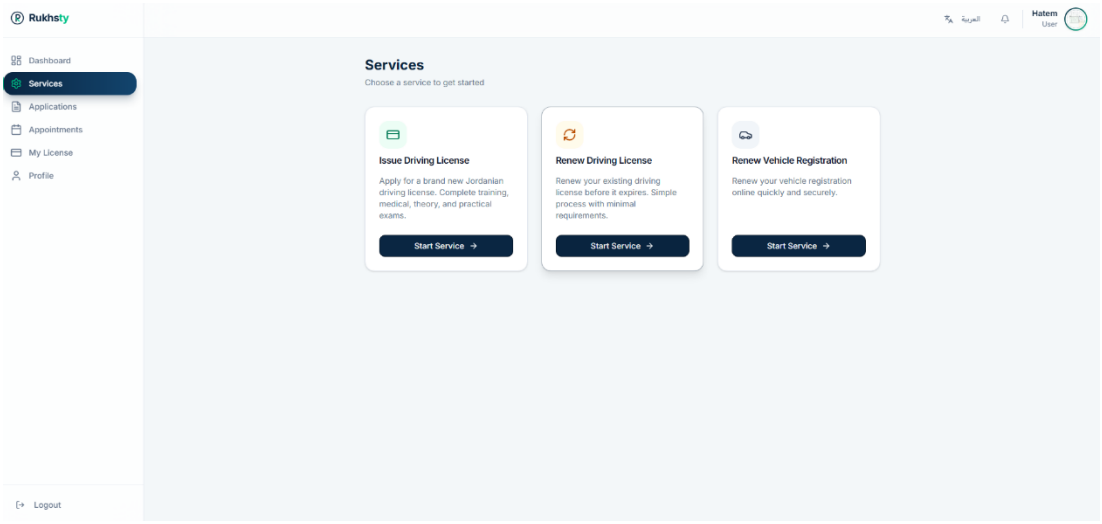


Figure 20. User Services Page

User Services Page

This page shows the services available for the user, such as issuing a license, renewing a license, and renewing vehicle registration.

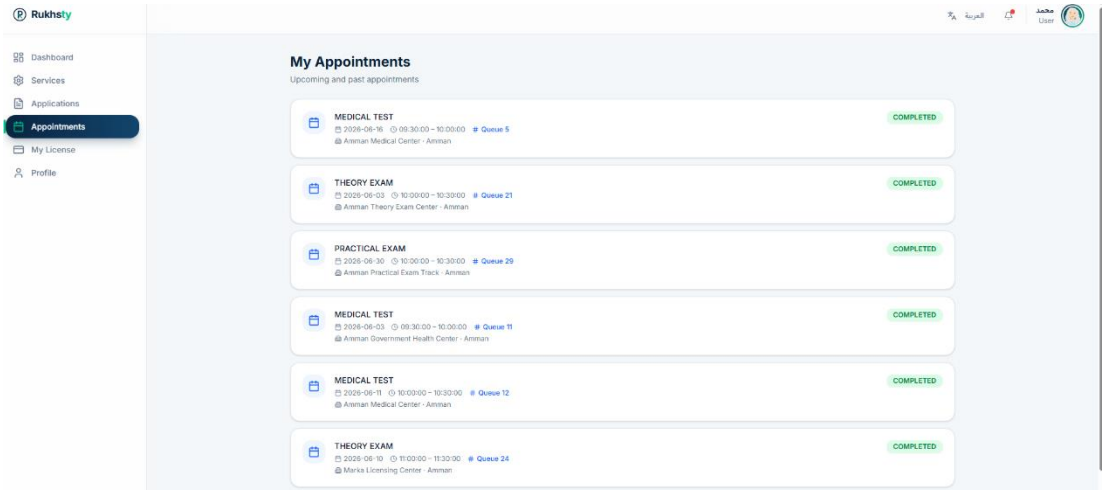


Figure 21. My Appointments Page

My Appointments Page

This page shows the user's booked appointments, including medical tests, theory exams, practical exams, dates, times, locations, and status**.**

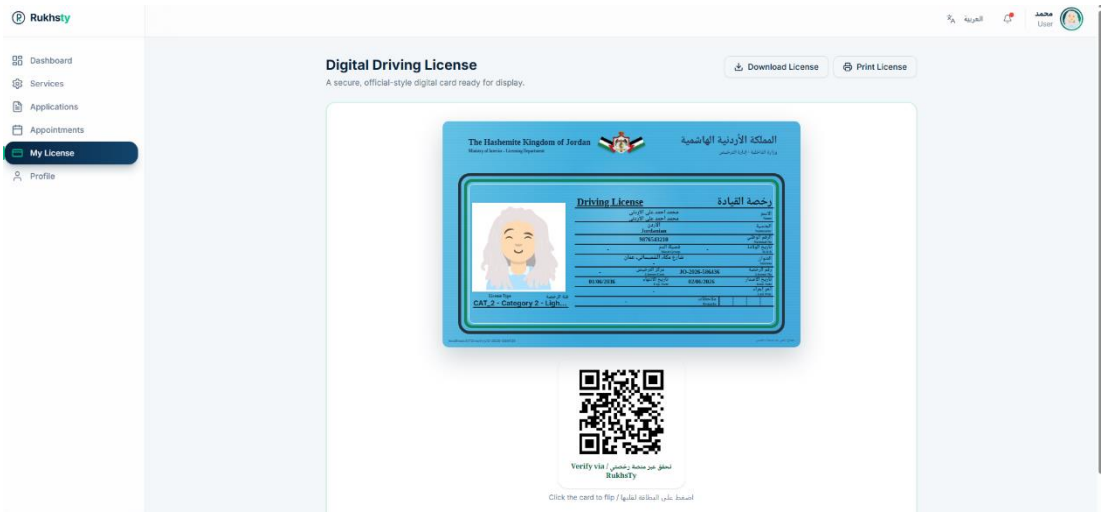


Figure 22. Digital Driving License Page

Digital Driving License Page

This page displays the issued digital driving license with a QR code, and options to download or print the license.

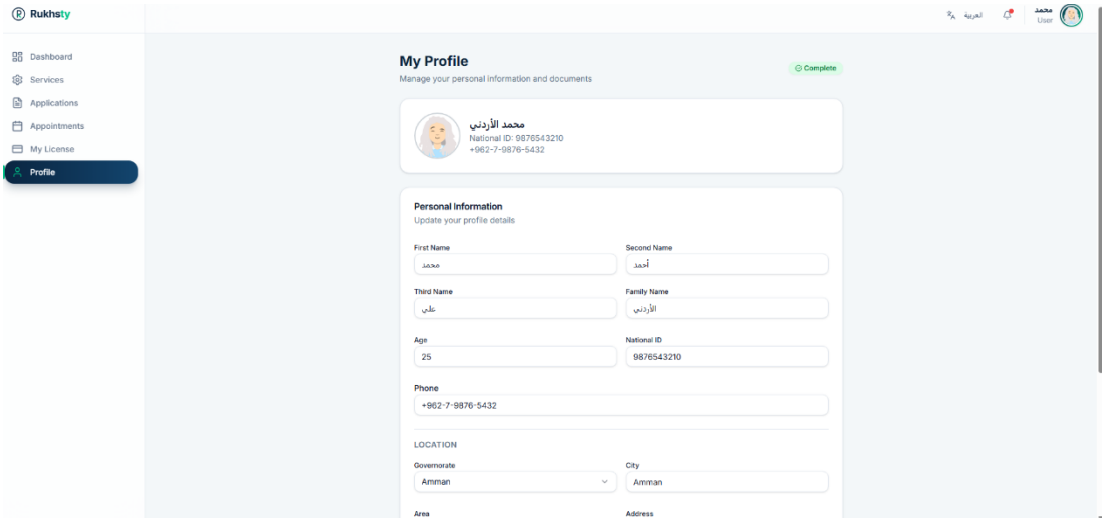


Figure 23. User Profile Page

User Profile Page

This page shows the user's personal information, contact details, and location details saved in the system

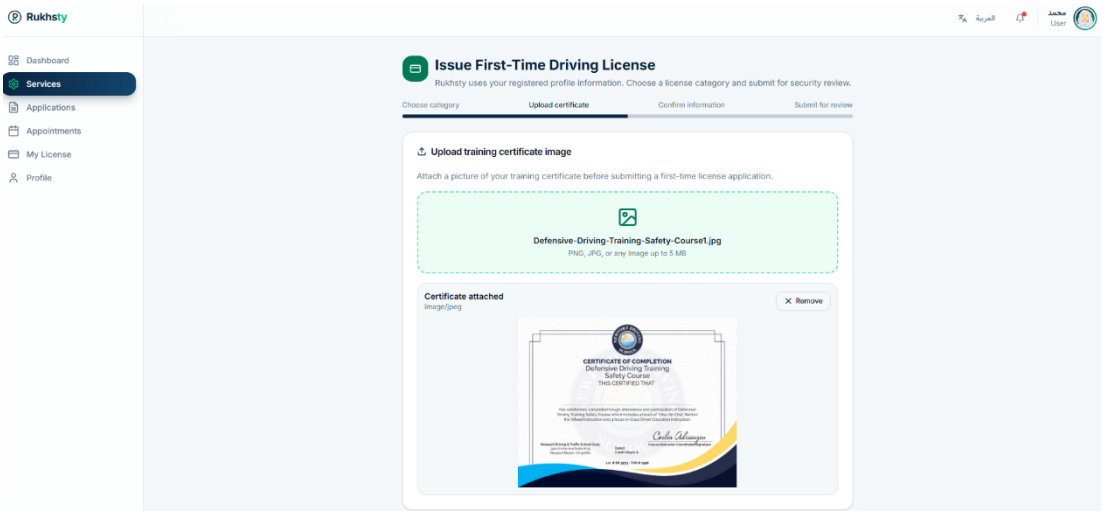


Figure 24. Upload Training Certificate

Upload Training Certificate

This page allows the user to upload the required training certificate as part of the first-time license application.

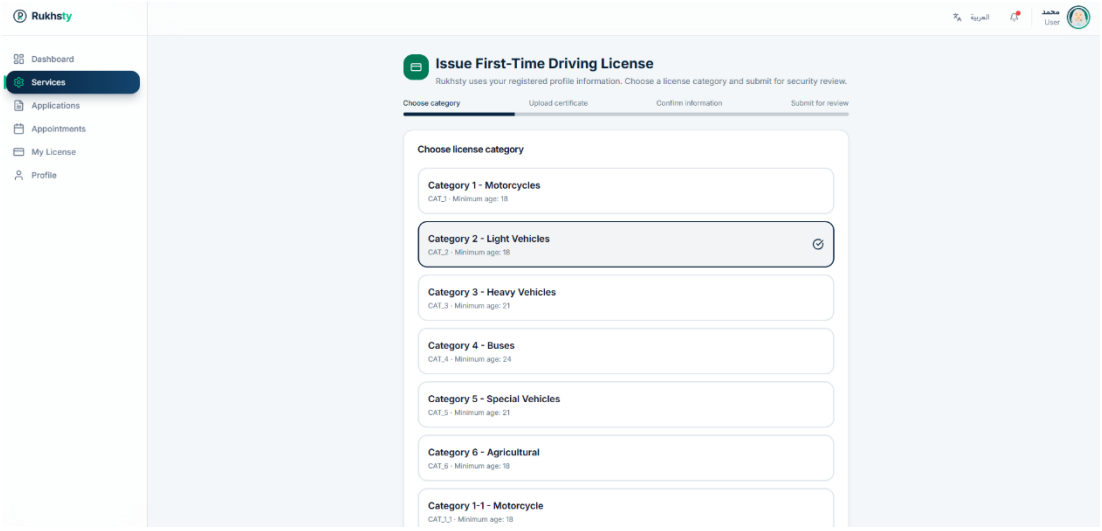


Figure 25. Choose License Category

Choose License Category

This page allows the user to select the required driving license category before continuing the application.

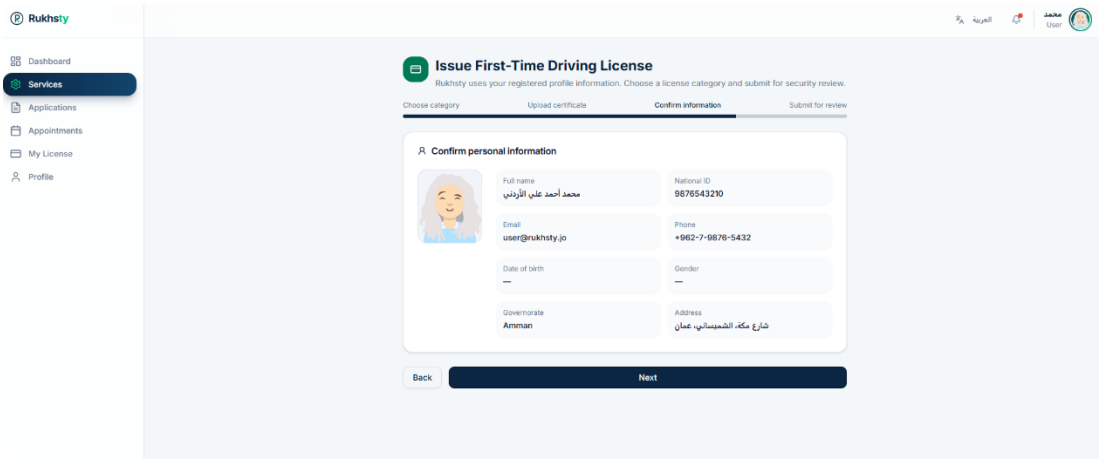


Figure 26. Confirm Personal Information

Confirm Personal Information

This page allows the user to review their personal information before submitting the application.

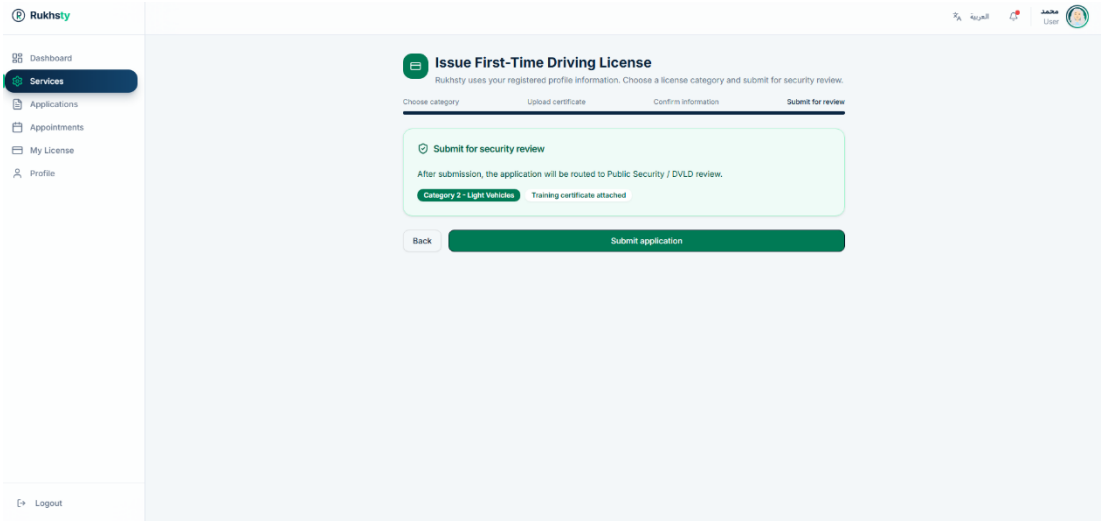


Figure 27. Submit for Security Review

Submit for Security Review

This page shows the final step before sending the application to the security review stage.

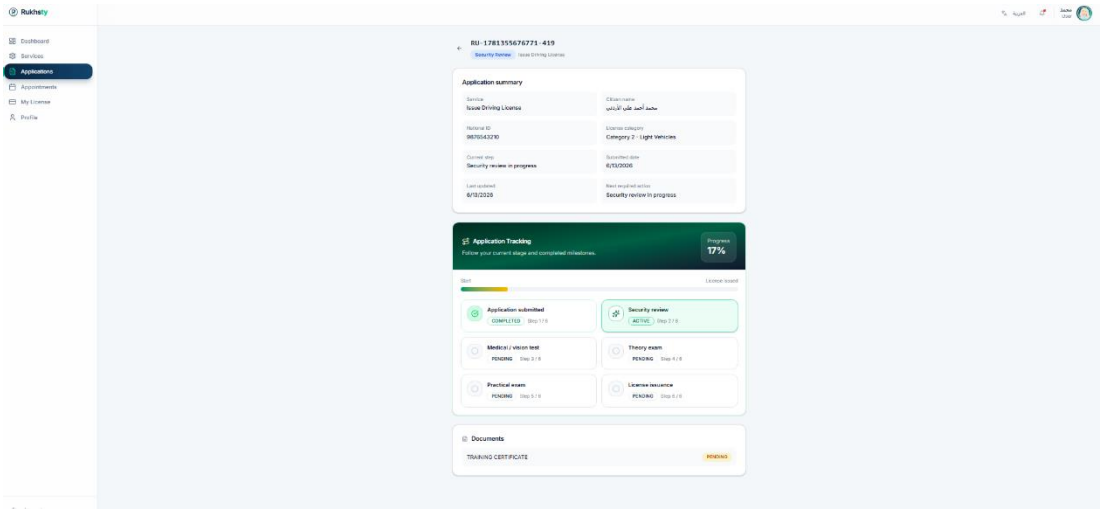


Figure 28. Application Tracking Page

Application Tracking Page

This page shows the current application status and progress through the licensing steps.

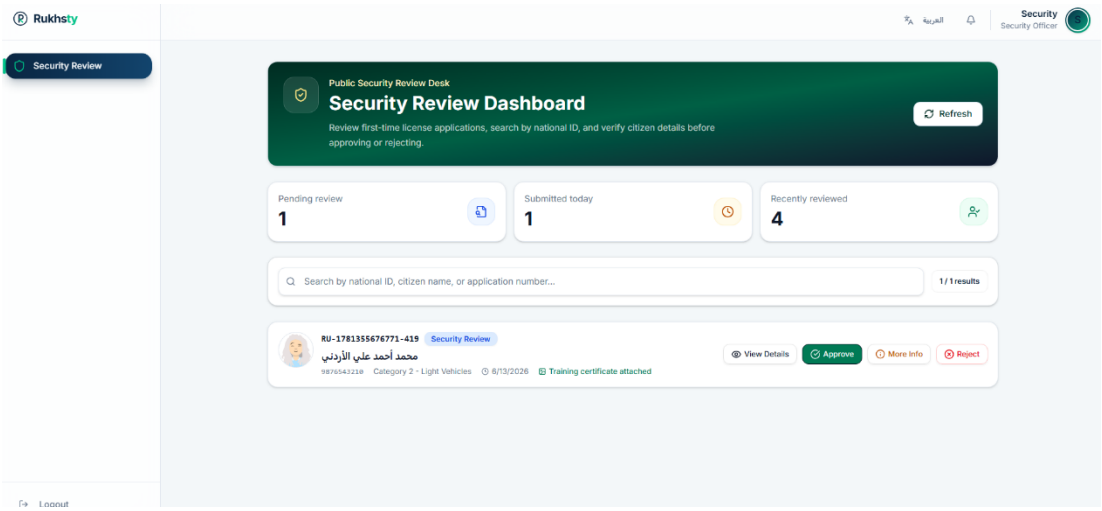


Figure 29. Security Review Dashboard

Security Review Dashboard

This page allows the security officer to view submitted applications and approve, reject, or request more information.

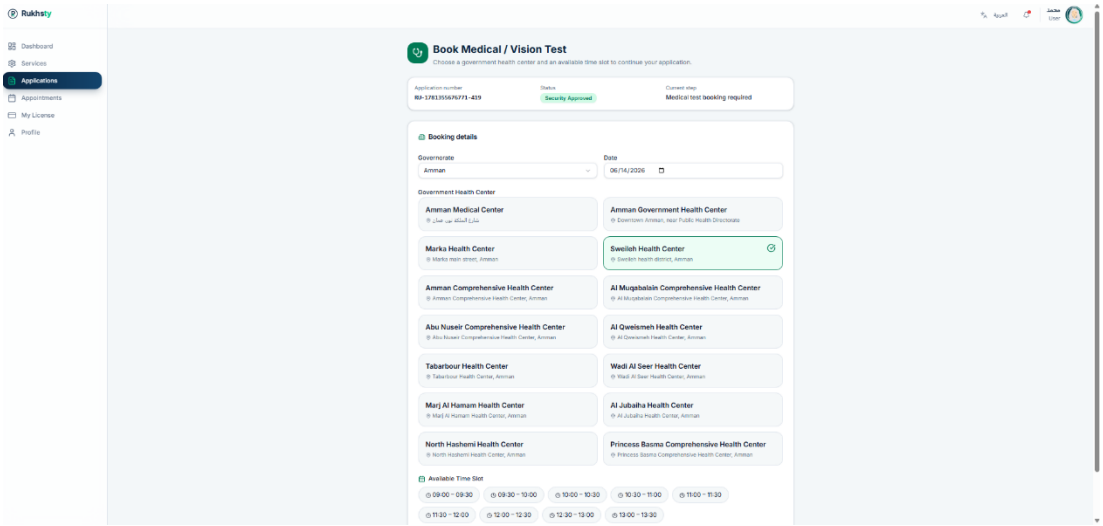


Figure 30. Book Medical / Vision Test

Book Medical / Vision Test

This page allows the user to choose a health center, date, and available time slot for the medical or vision test.

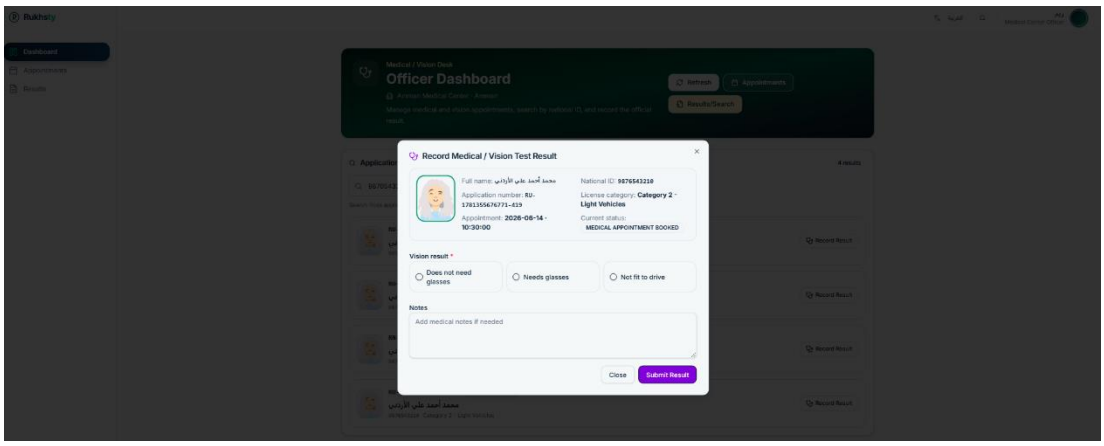


Figure 31. Medical / Vision Test Result

Medical / Vision Test Result

This page allows the medical officer to record the applicant's vision test result and add notes if needed.

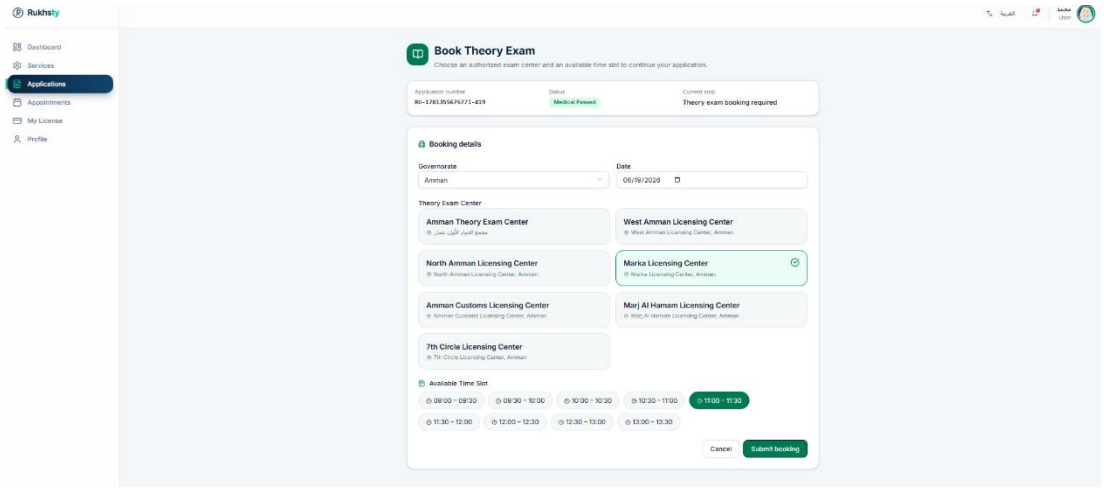


Figure 32. Book Theory Exam

Book Theory Exam

This page allows the user to choose a theory exam center, date, and available time slot.

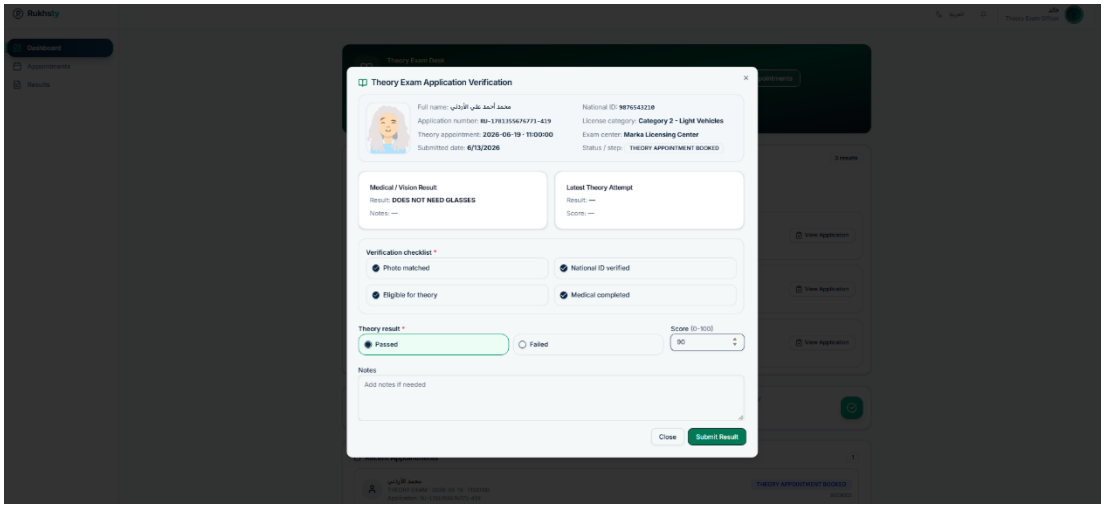


Figure 33. Theory Exam Verification

Theory Exam Verification

This page allows the theory officer to verify the applicant information and submit the theory exam result

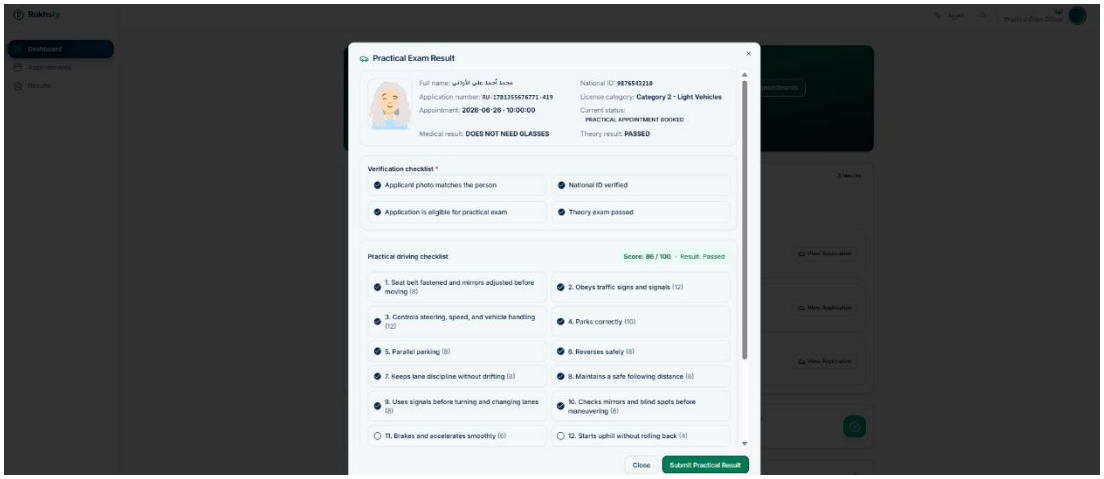


Figure 34. Practical Exam Result

Practical Exam Result

This page allows the practical officer to check the driving test requirements and submit the practical exam result.

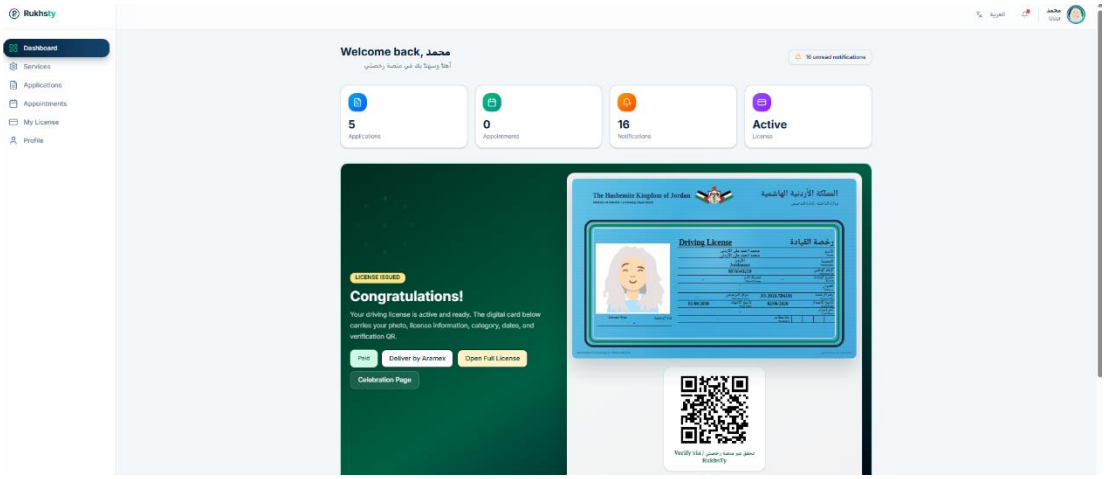


Figure 35. License Issued Dashboard

License Issued Dashboard

This page shows that the user's license is active and ready, with options to view the full license or request delivery.

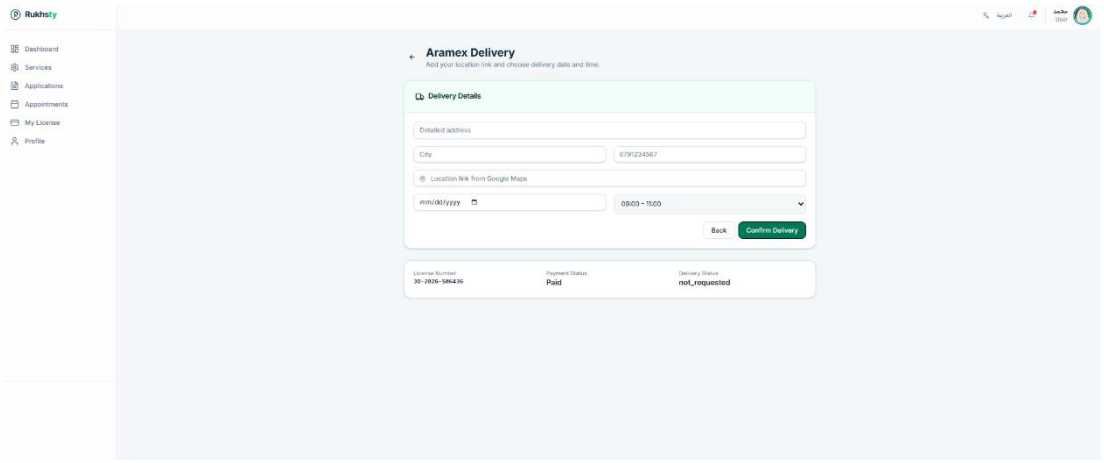


Figure 36. Aramex Delivery Page

Aramex Delivery Page

This page allows the user to enter delivery details and request license delivery through Aramex.

Chapter 5: System Implementation

5.1 Overview

This chapter describes the implementation of the RukhsTy system, including the deployment approach, development environment, main system components, and technologies used to build the application. RukhsTy was implemented as a full-stack web application that connects a responsive frontend interface with a backend API server and a relational database.

The system is designed to support multiple user roles, including Citizen/User, Admin, Security Officer, Medical Officer, Theory Officer, and Practical Officer. Each role has access to specific features based on its responsibilities in the driving license workflow. Citizens can register, log in, submit applications, upload required documents, track application status, pay through a mock eFAWATEERcom payment flow, view the digital license, and request mock Aramex delivery. Officers can review applications, record test results, and update application progress, while admins can manage users, applications, reports, and system data.

The implementation follows a client-server architecture. The frontend is responsible for presenting the user interface and collecting user input, while the backend handles authentication, validation, business logic, database operations, and API responses. The PostgreSQL database stores the main system data, including users, profiles, applications, documents, appointments, exams, licenses, payments, notifications, and delivery information.

The system can be deployed as a web application using a Node.js runtime environment. During development, the application is run locally using project commands, while the backend connects to the database through environment configuration. The deployment process requires installing dependencies, configuring environment variables, preparing the database schema, running migrations if needed, and starting the frontend and backend services.

The major implementation tasks included setting up the project structure, building the frontend pages, developing backend REST API endpoints, designing the database schema, implementing JWT-based authentication, applying role-based access control, connecting the frontend to backend APIs, testing the main workflows, and preparing the system for demonstration.

The resources required to support the implementation include a development machine, Node.js environment, pnpm package manager, PostgreSQL database, source code editor, GitHub repository for version control, and browser testing tools. The system does not require special hardware because it is designed as a standard web application that can run on a local development environment or be deployed to a cloud hosting platform.

5.2 Tools and Technologies Used

RukhsTy was implemented as a TypeScript monorepo using **pnpm** as the package manager. This structure allows the frontend, backend, and shared code to be organized inside one project, making development easier and improving consistency between client-side and server-side code.

The frontend was developed using **React** with **Vite**. React was selected because it supports reusable components, dynamic user interfaces, and efficient single-page application development. Vite was used because it provides fast development startup, quick hot module replacement, and a modern build process. Compared with heavier frontend frameworks, React with Vite was more suitable for RukhsTy because the project requires a fast, interactive, and dashboard-based user experience.

The routing system was implemented using **Wouter**, which provides lightweight client-side routing. This allows users to move between pages such as login, registration, citizen dashboard, application tracking, officer dashboards, digital license page, payment section, and delivery form without reloading the entire application.

For server-state management, **React Query** was used to handle API requests, caching, loading states, and response updates. This improved the communication between the frontend and backend and reduced repeated manual API handling. **React Hook Form** and **Zod** were used for form handling and validation. These tools helped validate user inputs such as registration data, login credentials, application forms, delivery information, and other required fields before sending requests to the backend.

The user interface was styled using **Tailwind CSS**, supported by **Radix UI components**, **Lucide icons**, and **Framer Motion**. Tailwind CSS was selected because it allows fast and consistent styling using utility classes. Radix UI helped provide accessible interface components, Lucide icons improved the visual clarity of buttons and menus, and Framer Motion was used to enhance the user experience with smooth animations.

The backend was implemented using **Express.js 5** with **TypeScript**. Express was selected because it is simple, flexible, and suitable for building REST API endpoints. Compared with more opinionated frameworks such as NestJS, Express was more appropriate for this project because RukhsTy's backend is route-focused and benefits from direct middleware composition. The backend was bundled using **esbuild**, which provides fast TypeScript compilation and efficient server builds.

The database used in RukhsTy is **PostgreSQL**. PostgreSQL was selected instead of a document-based database because RukhsTy has strong relational requirements. The system needs to manage relationships between citizens, user profiles, applications, documents, appointments, officers, exams, licenses, payments, notifications, and delivery requests. A relational database provides strong data consistency and supports structured queries for these connected entities.

Drizzle ORM was used to define database schemas and perform type-safe database access. Drizzle was selected instead of raw SQL-only implementation because it keeps the database schema aligned with TypeScript types while still allowing clear and explicit query logic. **Drizzle Zod** was also used to connect schema definitions with validation logic, improving consistency between database structure and application validation.

Authentication and authorization were implemented using **JWT tokens** and **bcryptjs**. JWT was used to manage secure user sessions and identify logged-in users. bcryptjs was used to hash user passwords before storing them in the database, ensuring that plain-text passwords are never saved. Role-based access control was applied to protect dashboards and backend endpoints so that each user can only access the features related to their role.

OpenAPI and **Orval** were used to support API definition and client generation. These tools help document backend endpoints and generate consistent API client code, reducing mismatch between frontend requests and backend routes.

For logging, **Pino** and **pino-http** were used to provide structured API logs. These logs help developers monitor backend requests, detect errors, and debug issues during development and testing.

Overall, the selected tools and technologies helped the team build a secure, maintainable, and scalable web application. The combination of React, TypeScript, Express.js, PostgreSQL, Drizzle ORM, JWT authentication, and modern frontend libraries provided a strong technical foundation for implementing the RukhsTy driving license management system.

5.2 Tools and Technologies Summary

No.	Tool / Technology	Usage in RukhsTy
1	TypeScript	Used for type-safe frontend, backend, and shared code development.
2	pnpm	Used as the package manager for the monorepo project structure.
3	React	Used to build the frontend user interface.
4	Vite	Used for fast frontend development and build process.
5	React Query	Used for API requests, caching, and server-state management.
6	Tailwind CSS	Used for responsive and consistent interface styling.
7	Radix UI	Used for accessible UI components.
8	Lucide Icons	Used for interface icons.
9	Framer Motion	Used for animations and smoother user experience.
10	Express.js	Used to build backend REST API endpoints.
11	esbuild	Used to bundle the TypeScript backend.
12	PostgreSQL	Used as the relational database.
13	Drizzle ORM	Used for database schema definition and type-safe queries.
14	Drizzle Zod	Used to connect database schemas with validation.
15	JWT	Used for authentication and secure session handling.
16	bcryptjs	Used for password hashing and password comparison.
17	Orval	Used for API client and validation code generation.
18	GitHub	Used for version control and team collaboration.

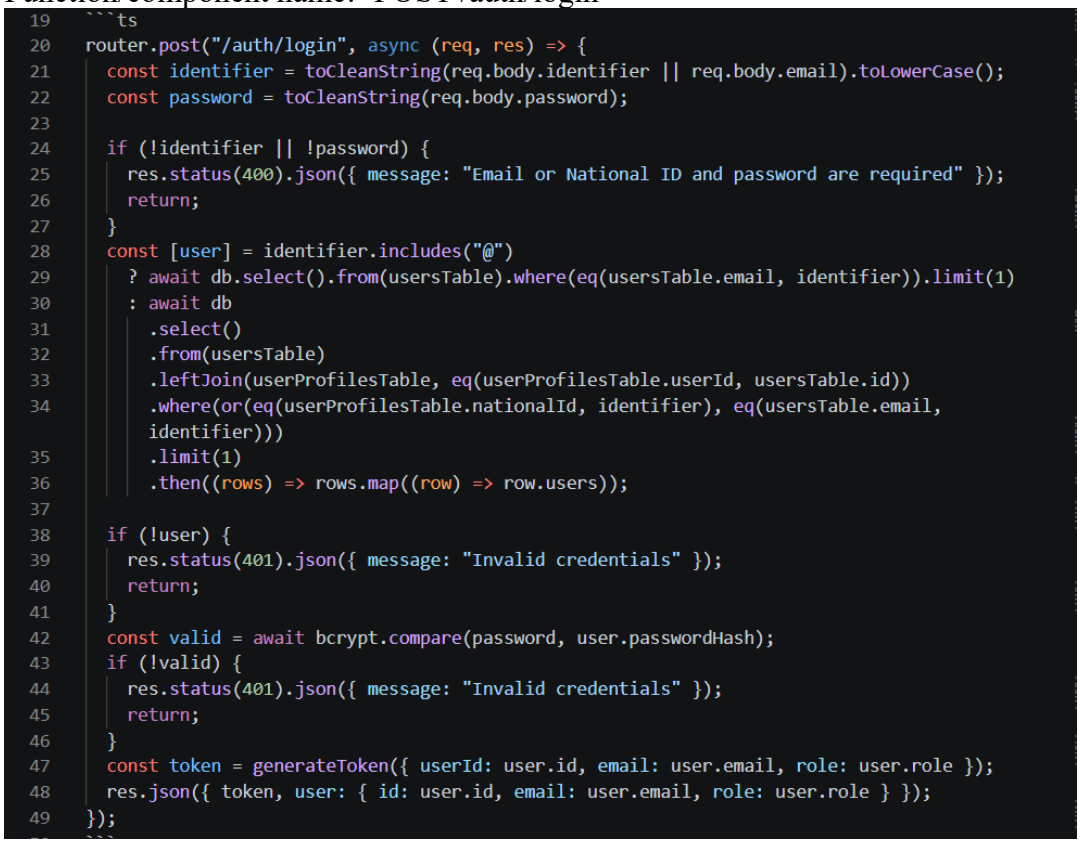
Table 4

5.3 Important Code Fragments

1. Authentication and Login Backend

File path: `artifacts/api-server/src/routes/auth.ts`

Function/component name: `POST /auth/login`



```ts
router.post("/auth/login", async (req, res) => {
  const identifier = toCleanString(req.body.identifier || req.body.email).toLowerCase();
  const password = toCleanString(req.body.password);

  if (!identifier || !password) {
    res.status(400).json({ message: "Email or National ID and password are required" });
    return;
  }
  const [user] = identifier.includes("@")
    ? await db.select().from(usersTable).where(eq(usersTable.email, identifier)).limit(1)
    : await db
      .select()
      .from(usersTable)
      .leftJoin(userProfilesTable, eq(userProfilesTable.userId, usersTable.id))
      .where(or(eq(userProfilesTable.nationalId, identifier), eq(usersTable.email,
      identifier)))
      .limit(1)
      .then((rows) => rows.map((row) => row.users));

  if (!user) {
    res.status(401).json({ message: "Invalid credentials" });
    return;
  }
  const valid = await bcrypt.compare(password, user.passwordHash);
  if (!valid) {
    res.status(401).json({ message: "Invalid credentials" });
    return;
  }
  const token = generateToken({ userId: user.id, email: user.email, role: user.role });
  res.json({ token, user: { id: user.id, email: user.email, role: user.role } });
});
```

Figure 37. Authentication and Login Backend

This route allows login using either email or national ID. It compares the submitted password against the stored bcrypt password hash and returns a JWT token containing the user ID, email, and role

2.Frontend Login and Role Redirection

File path: `artifacts/rukhsty/src/pages/public/login.tsx`

Function/component name: `Login.onSubmit`

```
const onSubmit = async (values: z.infer<typeof loginSchema>) => {
  try {
    const normalizedIdentifier = values.identifier.trim();
    const response = await loginMutation.mutateAsync({
      data: {
        identifier: normalizedIdentifier,
        email: normalizedIdentifier.includes("@") ? normalizedIdentifier : undefined,
        password: values.password,
      },
    });
    login(response.token, response.user);

    toast({
      title: t("loginSuccessTitle"),
      description: t("loginSuccessDescription"),
    });

    if (response.user.role === "SECURITY_OFFICER") {
      setLocation("/security/review");
    } else if (response.user.role === "ADMIN") {
      setLocation("/admin/dashboard");
    } else if (response.user.role.includes("OFFICER")) {
      setLocation("/officer/dashboard");
    } else {
      setLocation("/dashboard");
    }
  } catch {
    toast({
      variant: "destructive",
      title: t("loginFailedTitle"),
      description: t("loginFailedDescription"),
    });
```

Figure 38. Frontend Login and Role Redirection

This frontend login handler sends the identifier and password to the generated API client, stores the token through the auth context, and redirects users to dashboards based on their role.

3. User Registration Backend

File path: `artifacts/api-server/src/routes/auth.ts`

Function/component name: `POST /auth/register`

```
router.post("/auth/register", async (req, res) => {
  const email = toCleanString(req.body.email).toLowerCase();
  const password = toCleanString(req.body.password);
  const firstName = toCleanString(req.body.firstName);
  const secondName = toCleanString(req.body.secondName);
  const thirdName = toCleanString(req.body.thirdName);
  const familyName = toCleanString(req.body.familyName);
  const nationalId = toCleanString(req.body.nationalId);
  const phone = toCleanString(req.body.phone);
  const governorate = toCleanString(req.body.governorate);
  const city = toCleanString(req.body.city);
  const area = toCleanString(req.body.area);
  const address = toCleanString(req.body.address);
  const dateOfBirth = toCleanString(req.body.dateOfBirth);
  const gender = toCleanString(req.body.gender);
  const personalPhotoUrl = toCleanString(req.body.personalPhotoUrl);
  const idFrontUrl = toCleanString(req.body.idFrontUrl);
  const idBackUrl = toCleanString(req.body.idBackUrl);

  if (!email || !password || !firstName || !secondName || !thirdName || !familyName ||
  !nationalId || !phone || !dateOfBirth || !gender || !governorate || !address ||
  !personalPhotoUrl) {
    res.status(400).json({ message: "Required registration fields are missing" });
    return;
  }
  if (!email.includes("@")) {
    res.status(400).json({ message: "A valid email address is required" });
    return;
  }
  if (!nationalIdPattern.test(nationalId)) {
    res.status(400).json({ message: "National ID must be 10 digits" });
    return;
```

Figure 39. User Registration Backend

This first part of the registration route cleans request values and validates required fields, email format, national ID format, and password strength before creating any database rows.

4.Database Schema / Model Layer

File path: `lib/db/src/schema/users.ts`

Function/component name: `usersTable`

```
export const userRoleEnum = pgEnum("user_role", [
  "USER",
  "ADMIN",
  "TRAINING_CENTER_OFFICER",
  "MEDICAL_OFFICER",
  "MEDICAL_CENTER_OFFICER",
  "THEORY_OFFICER",
  "THEORY_EXAM_OFFICER",
  "PRACTICAL_OFFICER",
  "PRACTICAL_EXAM_OFFICER",
  "DVLD_OFFICER",
  "SECURITY_OFFICER",
]);

export const usersTable = pgTable("users", {
  id: uuid("id").primaryKey().defaultRandom(),
  email: text("email").notNull().unique(),
  passwordHash: text("password_hash").notNull(),
  role: userRoleEnum("role").notNull().default("USER"),
  isEmailVerified: boolean("is_email_verified").notNull().default(false),
  isActive: boolean("is_active").notNull().default(true),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
});
```

Figure 40. Database Schema / Model Layer

This schema defines system users, including role values used by backend and frontend guards.

5.Submit First-Time Driving License Application

File path: `artifacts/api-server/src/routes/applications.ts`

Function/component name: `POST /applications`

```
router.post("/applications", requireAuth, async (req, res) => {
  const { userId } = (req as Request & { user: JwtPayload }).user;
  const { serviceId, licenseCategoryId, governorate, residenceArea, trainingCertificate } =
  req.body;
  const [service] = serviceId ? await db.select().from(servicesTable).where(eq(servicesTable.
  id, serviceId)).limit(1) : [null];
  const isIssueDrivingLicense = service?.code === "ISSUE_DRIVING_LICENSE";
  if (isIssueDrivingLicense) {
    const fileUrl = typeof trainingCertificate?.fileUrl === "string" ? trainingCertificate.
    fileUrl.trim() : "";
    const fileName = typeof trainingCertificate?.fileName === "string" ? trainingCertificate.
    fileName.trim() : "";
    const mimeType = typeof trainingCertificate?.mimeType === "string" ? trainingCertificate.
    mimeType.trim() : "";
    if (!fileUrl || !fileName || !mimeType.startsWith("image/")) {
      res.status(400).json({ message: "Training certificate image is required for first-time
      driving license applications" });
      return;
    }
  }
  const appNumber = `RU-${Date.now()}-${Math.floor(Math.random() * 1000)}`;
  const [app] = await db.insert(applicationsTable).values({
    userId,
    serviceId,
    licenseCategoryId,
    applicationNumber: appNumber,
    status: isIssueDrivingLicense ? "SECURITY_REVIEW" : "DRAFT",
    currentStep: isIssueDrivingLicense ? "SECURITY_REVIEW" : "PROFILE_REVIEW",
    governorate,
    residenceArea,
    submittedAt: isIssueDrivingLicense ? new Date() : null,
  }).returning();
```

Figure 41. Submit First-Time Driving License Application

This route creates a citizen application and requires a training certificate image when the selected service is first-time driving license issuance.

6. Security Officer Workflow

File path: `artifacts/api-server/src/routes/admin.ts`

Function/component name: `GET /admin/applications`

```
router.get("/admin/applications", requireAuth, requireRole("ADMIN", "DVLD_OFFICER",
"SECURITY_OFFICER"), async (req, res) => {
  const { status, serviceCode, page = "1", limit = "20" } = req.query as { status?: string;
  serviceCode?: string; page?: string; limit?: string };
  const offset = (Number(page) - 1) * Number(limit);
  const conditions = [];
  if (status && status !== "ALL") conditions.push(eq(applicationsTable.status, status));
  if (serviceCode) {
    const [service] = await db.select().from(servicesTable).where(eq(servicesTable.code,
    serviceCode)).limit(1);
    if (!service) {
      res.json({ data: [], page: Number(page), limit: Number(limit), total: 0 });
      return;
    }
    conditions.push(eq(applicationsTable.serviceId, service.id));
  }
  const query = db.select().from(applicationsTable);
  const apps = conditions.length
    ? await query.where(and(...conditions)).orderBy(desc(applicationsTable.createdAt)).limit
    (Number(limit)).offset(offset)
    : await query.orderBy(desc(applicationsTable.createdAt)).limit(Number(limit)).offset
    (offset);
  const data = await Promise.all(apps.map(enrichApplication));
  res.json({ data, page: Number(page), limit: Number(limit), total: data.length });
});
```

Figure 42. Security Officer Workflow

This route lists applications for admin, DVLD, and security officers. The frontend uses it with `status=SECURITY_REVIEW` and `serviceCode=ISSUE_DRIVING_LICENSE`.

7. Medical Officer Workflow

File path: `artifacts/api-server/src/routes/officer.ts`

Function/component name: `recordMedical`

```
async function recordMedical(req: Request, res: Response) {
  if (!assertRole(req, res, ["MEDICAL_OFFICER", "MEDICAL_CENTER_OFFICER", "ADMIN"])) return;
  const { userId: officerId } = (req as Request & { user: JwtPayload }).user;
  const { applicationId = routeParam(req, "applicationId"), centerId, result, leftEyeScore,
  rightEyeScore, requiresGlasses, isAllowedToDrive, notes } = req.body;
  const allowedResults = ["DOES_NOT_NEED_GLASSES", "NEEDS_GLASSES", "NOT_FIT_TO_DRIVE",
  "APPROVED_NO_GLASSES", "APPROVED_NEEDS_GLASSES", "NOT_APPROVED_NOT_FIT",
  "PASS_NO_GLASSES", "PASS_WITH_GLASSES"];
  if (!allowedResults.includes(result)) {
    res.status(400).json({ message: "A valid medical result is required" });
    return;
  }
```

Figure 43. Medical Officer Workflow

This function allows only medical officers or admins to record vision/medical results and validates allowed result values

8. Digital License Issuing

File path: `artifacts/api-server/src/services/license-issuance.ts`

Function/component name: `issueLicenseForApplication`

```
export async function issueLicenseForApplication(applicationId: string, _issuedBy?: string |
null) {
  const [app] = await db.select().from(applicationsTable).where(eq(applicationsTable.id,
  applicationId)).limit(1);
  if (!app) throw Object.assign(new Error("Application not found"), { statusCode: 404 });

  const isEligible = ELIGIBLE_STATUSES.includes(app.status) || ELIGIBLE_STEPS.includes(app.
  currentStep ?? "");
  if (!isEligible) throw Object.assign(new Error("Application is not eligible for license
  issuance"), { statusCode: 409 });

  const [existingByApplication] = await db.select().from(drivingLicensesTable).where(eq
  (drivingLicensesTable.applicationId, applicationId)).limit(1);
  if (existingByApplication) {
    await completeIssuedApplication(app.id);
    return existingByApplication;
  }
```

Figure 44. Digital License Issuing

This service checks whether an application can receive a license and prevents duplicate license creation for the same application.

9. License issuance eligibility and duplicate prevention.**

File path: `artifacts/api-server/src/services/license-issuance.ts`

Function/component name: `issueLicenseForApplication`

```
const [license] = await db.insert(drivingLicensesTable).values({
  userId: app.userId,
  applicationId: app.id,
  licenseNumber,
  nationalId: profile?.nationalId ?? "UNKNOWN",
  fullNameAr: profileFullName(profile ?? null),
  fullNameEn: profileFullName(profile ?? null),
  licenseCategoryId: app.licenseCategoryId,
  issueDate: toIsoDate(issueDate),
  expiryDate: toIsoDate(expiryDate),
  status: "ACTIVE",
  photoUrl: profile?.personalPhotoUrl,
  qrCodeUrl: qrPayload,
}).returning();

await completeIssuedApplication(app.id);
await db.insert(notificationsTable).values({
  userId: app.userId,
  title: "License issued",
  message: "Congratulations! You passed the practical exam and received your driving
  license. مبروك! لقد اجتزت الاختبار العملي وحصلت على الرخصة.",
  type: "SUCCESS",
});
return license;
```

Figure 45. License Issuance Eligibility and Duplicate Prevention

This code creates the driving license record, stores the QR payload, completes the application, and notifies the citizen.

10. Mock eFAWATEERcom Payment

File path: `artifacts/api-server/src/routes/licenses.ts`

Function/component name: `generateEfawateercomReference`

```
export async function generateEfawateercomReference() {
  for (let i = 0; i < 8; i += 1) {
    const reference = `RKH-${todayCompact()}-${Math.floor(100000 + Math.random() * 900000)}`;
    const [existingLicense] = await db.select().from(drivingLicensesTable).where(eq
    (drivingLicensesTable.paymentReference, reference)).limit(1);
    const [existingApplication] = await db.select().from(applicationsTable).where(eq
    (applicationsTable.paymentReference, reference)).limit(1);
    if (!existingLicense && !existingApplication) return reference;
  }
  return `RKH-${todayCompact()}-${String(Date.now()).slice(-6)}`;
}
```

Figure 46. Mock eFAWATEERcom Payment

This helper generates a mock eFAWATEERcom-style payment reference and checks both licenses and applications to avoid collisions.

11. Frontend Route Protection and Component Structure

File path: `artifacts/rukhsty/src/Router.tsx`

Function/component name: `PublicRoute`, `ProtectedRoute`

```
function PublicRoute({ component: Component }: { component: React.ComponentType }) {
  const { isAuthenticated, user, isLoading } = useAuth();
  if (isLoading) return <div className="min-h-screen flex items-center justify-center
  bg-background"><div className="w-8 h-8 border-2 border-primary border-t-transparent
  rounded-full animate-spin" /></div>;
  if (isAuthenticated && user) {
    if ((user as any).role === "SECURITY_OFFICER") return <Redirect to="/security/review" />;
    if ((user as any).role === "ADMIN") return <Redirect to="/admin/dashboard" />;
    if ((user as any).role?.includes("OFFICER")) return <Redirect to="/officer/dashboard" />;
    return <Redirect to="/dashboard" />;
  }
  return <Component />;
}

function ProtectedRoute({ component: Component, allowedRoles = [], params }: { component:
React.ComponentType<any>; allowedRoles?: string[]; params?: Record<string, string |
undefined> }) {
  return (
    <AppShell requireAuth allowedRoles={allowedRoles}>
      <Component params={params} />
    </AppShell>
  );
}
```

Figure 47. Frontend Route Protection and Component Structure

These frontend wrappers redirect already-authenticated users and protect private pages through `AppShell` and role lists.

Chapter 6: Testing and Validation

Testing and validation were performed to ensure that the Rukhsty system meets its functional and non-functional requirements. The testing process focused on verifying user authentication, application submission, officer workflows, payment simulation, digital license issuing, delivery request handling, and application status tracking. The system was tested using black-box testing, white-box testing, and manual/automated validation techniques. Black-box testing was used to test the system from the user perspective without checking the internal code. White-box testing was used to verify important backend logic, validation rules, authentication, role guards, and application status transitions

6.1 Black-Box

Black-box testing focuses on testing the visible system behavior based on inputs and expected outputs. The tester does not need to know the internal source code. In Rukhsty, black-box testing was applied to login, registration, application submission, certificate upload, officer approval, appointment booking, mock payment, digital license display, QR verification, and delivery request

Test Table

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/ Fail
TC01	Check citizen login with valid data	1. Open login page. 2. Enter email/national ID. 3. Enter password. 4. Click Login.	Email/National ID: valid user Password: valid password	User should log in and be redirected to citizen dashboard.	As expected	Pass
TC02	Check login with invalid password	1. Open login page. 2. Enter valid email/national ID. 3. Enter wrong password. 4. Click Login.	Email/National ID: valid user Password: wrong password	System should reject login and show an error message.	As expected	Pass
TC03	Register a new citizen account	1. Open registration page. 2. Enter required personal information. 3. Enter email and password. 4. Submit the form.	New email, national ID, name, password, profile data	System should create a new user account successfully.	As expected	Pass
TC04	Register using existing email or national ID	1. Open registration page. 2. Enter an email or national ID already used. 3. Submit the form.	Existing email or national ID	System should prevent registration and display an error message.	As expected	Pass
TC05	Submit license application with all required data	1. Log in as citizen. 2. Open issue license service. 3. Fill application details. 4. Upload training certificate. 5. Submit application.	Valid application data and certificate image	Application should be submitted and sent to security review.	As expected	Pass
TC06	Submit license application without certificate	1. Log in as citizen. 2. Open issue license service. 3. Fill application data. 4. Do not upload certificate. 5. Submit application.	Missing certificate	System should block submission and show validation message.	As expected	Pass
TC07	Track application status	1. Log in as citizen. 2. Open My Applications or Track Status page. 3. Select submitted application.	Existing submitted application	System should display current application status and progress.	As expected	Pass
TC10	Security officer rejects application	1. Log in as security officer. 2. Open pending application. 3. Click Reject and enter reason.	Pending application with rejection reason	Application status should become rejected.	As expected	Pass

TC11	Medical officer records medical test result	1. Log in as medical officer. 2. Open medical dashboard. 3. Select applicant. 4. Record pass/fail result.	Applicant in medical stage	System should save result and update application status.	As expected	Pass
TC12	Theory officer records theory exam result	1. Log in as theory officer. 2. Open theory dashboard. 3. Select applicant. 4. Record pass/fail result.	Applicant in theory stage	System should save theory result and update application status.	As expected	Pass
TC13	Practical officer records practical exam result	1. Log in as practical officer. 2. Open practical dashboard. 3. Select applicant. 4. Record pass/fail result.	Applicant in practical stage	System should save practical result and move application to license issuing if passed.	As expected	Pass
TC14	Pay by mock eFAWATEERcom	1. Log in as citizen. 2. Open approved/issued license page. 3. Click Pay by eFAWATEERcom.	Approved unpaid license	Payment status should become paid and license should be available.	As expected	Pass
TC15	View digital license	1. Log in as citizen. 2. Open digital license page.	Issued and paid license	Digital license card and QR code should be displayed.	As expected	Pass
TC16	Request Aramex delivery	1. Open delivery page. 2. Enter address, city, phone, location link, date, and time slot. 3. Submit request.	Valid delivery data	System should confirm delivery request and generate tracking details.	As expected	Pass
TC17	Request delivery with missing data	1. Open delivery page. 2. Leave required field empty. 3. Submit request.	Missing address or phone	System should show validation message and prevent submission.	As expected	Pass

Table 5

6.2 White-Box

White-box testing focuses on testing the internal logic and code structure of the system. In Rukhsty, white-box testing was used to verify backend functions, database queries, authentication middleware, role-based guards, validation rules, and application workflow transitions.
The main techniques used were:

1. **Statement Coverage:**
   Important backend routes and functions were executed to ensure that each main statement was tested at least once.
2. **Branch Coverage:**
   Conditional statements such as valid/invalid login, approved/rejected application, pass/fail exam result, and paid/unpaid license were tested.
3. **Path Coverage:**
   Main workflow paths were tested, including successful application submission, rejection path, payment path, and delivery request path.
4. **Condition Testing:**
   Validation conditions were tested, such as checking if the certificate exists, checking user role, checking application status, and checking duplicate email or national ID.
5. **Security Testing:**
   Protected routes were tested to ensure that only authorized users with the correct role can access admin and officer functions.

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/ Fail
WB01	Authentication Function	Validate correct login credentials	If email/national ID exists and password matches hashed password	JWT token is generated and user data is returned.	As expected	Pass
WB02	Authentication Function	Validate wrong password	If password comparison fails	Login request returns error message.	As expected	Pass
WB03	Registration Function	Check duplicate email	If entered email already exists in database	System returns conflict/duplicate email error.	As expected	Pass
WB04	Registration Function	Check duplicate national ID	If national ID already exists in database	System returns conflict/duplicate national ID error.	As expected	Pass
WB05	Password Hashing Logic	Save new user password	Before saving user, password is hashed using bcrypt	Plain password is not stored in database.	As expected	Pass
WB06	Role Guard Middleware	Access admin dashboard as normal citizen	If user role is not Admin	Access should be denied.	As expected	Pass
WB07	Role Guard Middleware	Access security review as security officer	If user role is Security Officer	Access should be allowed.	As expected	Pass
WB08	Application Submission Logic	Submit application without certificate	If required certificate field is missing	Backend rejects request and returns validation error.	As expected	Pass
WB09	Application Workflow Logic	Security approval	If security officer approves application	Application status changes to the next workflow stage.	As expected	Pass
WB10	Application Workflow Logic	Security rejection	If security officer rejects application	Application status becomes rejected and reason is saved.	As expected	Pass
WB11	Medical Result Logic	Medical test passed	If result is pass	Application status moves to theory exam stage.	As expected	Pass
WB12	Theory Result Logic	Theory exam passed	If result is pass	Application status moves to practical exam stage.	As expected	Pass
WB13	Practical Result Logic	Practical exam passed	If result is pass	Application becomes ready for license issuing.	As expected	Pass
WB14	Mock Payment Endpoint	Pay unpaid license	If license exists and payment status is unpaid	Payment status becomes paid.	As expected	Pass

Table 6

6.3 Testing Automation

Testing automation was used to reduce repeated manual testing and to verify important system functions more quickly. Since Rukhsty is a graduation demo system, the main automation focused on API testing, type checking, and repeated validation of important backend endpoints.

The following testing automation activities were used:

1. **TypeScript Type Checking:**
   TypeScript checking was used to detect syntax errors, type mismatches, missing fields, and incorrect function usage before running the application.

2. **API Endpoint Testing:**
   Important backend endpoints such as login, registration, application submission, mock payment, and delivery request were tested using HTTP requests.

3. **Manual Browser Validation with Repeated Scenarios:**
   The main user workflows were repeatedly tested in the browser to confirm that UI pages interact correctly with backend APIs.

4. **Database Validation:**
   After important actions such as registration, application submission, payment, and delivery request, database records were checked to confirm that the correct data was saved.

Regression Testing:
After adding new features such as mock eFAWATEERcom payment and Aramex delivery, older workflows such as login, application tracking, and officer approval were retested to ensure they were not broken.

Automated / Repeated Testing Activities

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/ Fail
AT01	Type Checking	TypeScript / project command	Run type checking to detect code errors.	No type errors in important project files.	Pass	AT01
AT02	Login API	HTTP request / API client	Send login request with valid credentials.	API returns success response and token.	Pass	AT02
AT03	Invalid Login API	HTTP request / API client	Send login request with wrong password.	API returns authentication error.	Pass	AT03
AT04	Registration API	HTTP request / API client	Send registration request with new user data.	User account is created successfully.	Pass	AT04
AT05	Duplicate Registration API	HTTP request / API client	Send registration request using existing email or national ID.	API returns duplicate data error.	Pass	AT05
AT06	Application Submission API	HTTP request / browser workflow	Submit application with certificate upload.	Application is created and saved.	Pass	AT06
AT07	Security Approval API	HTTP request / officer dashboard	Approve a pending application.	Application status updates correctly.	Pass	AT07
AT08	Mock Payment API	HTTP request / citizen dashboard	Trigger mock eFAWATEERcom payment.	Payment status becomes paid.	Pass	AT08
AT09	Delivery API	HTTP request / delivery form	Submit Aramex delivery request.	Delivery data is saved and confirmation is returned.	Pass	AT09
AT10	Regression Testing	Manual repeated workflow	Retest login, tracking, approval, payment, and delivery after updates.	Existing features continue working correctly.	Pass	AT10

Table 7

Chapter 7: Conclusion and Future Work

7.1 Conclusion

Rukhsty is a digital driving license management system designed to simplify and organize the process of applying for driving license services in Jordan. The project successfully achieved its main objective by providing a complete web-based platform that connects citizens, administrators, and officers in one system.

The system allows citizens to register, log in, submit driving license applications, upload required documents such as the training certificate, track application status, book appointments, pay using a mock eFAWATEERcom payment flow, view the issued digital license, and request delivery through a mock Aramex delivery service. These features help reduce paperwork, improve transparency, and make the licensing process easier and faster for users.

Rukhsty also supports different officer roles, including Security Officer, Medical Officer, Theory Officer, and Practical Officer. Each officer has a dedicated workflow that supports the licensing process step by step. The Security Officer reviews submitted applications and approves or rejects them, while the Medical, Theory, and Practical Officers manage their related test stages and update the application progress.

From the technical side, the system was implemented using modern web technologies such as React, TypeScript, Express.js, PostgreSQL, Drizzle ORM, JWT authentication, and role-based access control. The system follows a client-server architecture, where the frontend communicates with the backend through REST API endpoints. This structure improves maintainability, scalability, and separation of responsibilities between the user interface, backend logic, and database layer.

The project also implemented important demo-friendly features such as mock eFAWATEERcom payment and mock Aramex delivery. These features were designed safely without using real external payment or delivery integrations, making the system suitable for academic demonstration while still showing a realistic licensing workflow.

Testing and validation confirmed that the main system functions work correctly, including registration, login, application submission, certificate validation, officer approval, medical/theory/practical workflows, payment simulation, digital license issuing, QR verification, and delivery request handling. Both black-box and white-box testing were used to verify the system behavior and internal logic.

Overall, Rukhsty successfully demonstrates how digital transformation can improve government service workflows by making them more accessible, organized, and user-friendly. The project meets its objectives and provides a strong foundation for future improvements and real-world deployment.

7.2 Future Work

Although RukhsTy successfully provides the main features required for the graduation project, several improvements can be added in future versions to make the platform more complete, secure, and ready for real-world deployment. One important improvement is connecting the system with the real eFAWATEERcom payment service instead of using a mock payment flow, allowing citizens to complete official payments securely. The delivery feature can also be improved by integrating with the official Aramex API to create real shipments, track delivery status, and notify users about delivery updates. In addition, the system can support SMS and email notifications to inform citizens about application updates, appointment reminders, payment confirmation, approval or rejection decisions, and delivery progress. Future versions may also include national identity verification to validate citizen information and reduce fake or duplicate accounts. The admin dashboard can be enhanced with advanced reports, charts, officer performance statistics, application completion rates, rejected application analysis, and payment summaries. The QR code verification feature can be improved to allow authorized officers to scan and verify digital licenses in real time. Furthermore, a mobile application for Android and iOS can be developed to make the service more accessible to citizens. The system can also be expanded with real-time appointment scheduling, cancellation, rescheduling, audit logging, security monitoring, cloud deployment, and automated end-to-end testing using tools such as Playwright or Cypress. These improvements would help transform RukhsTy from an academic prototype into a more complete and production-ready digital government service platform.

References

[1] World Bank. (n.d.). Digital Public Infrastructure and Services. World Bank Group.
[2] OECD. (n.d.). OECD Digital Government Studies. Organisation for Economic Co-operation and Development.
[3] Ministry of Digital Economy and Entrepreneurship. (2025). Jordanian Digital Transformation Strategy and Implementation Plan 2026–2028. Government of Jordan.
[4] Government of Jordan. (n.d.). Jordan Government Services Portal / Sanad Digital Services.
[5] Drivers and Vehicles Licensing Department. (n.d.). Electronic portal services for licensing management. Government of Jordan.
[6] Saudi National Platform. (n.d.). Driving license issuance. Kingdom of Saudi Arabia.
[7] Ministry of Interior, United Arab Emirates. (n.d.). Print driving license service.
[8] Silva, J. M., Ribeiro, D., Ramos, L. F., & Fonte, V. (2023). A worldwide overview on the information security posture of online public services.

Appendix

The appendix contains extra materials that support the RukhsTy graduation project report. These materials are not part of the main explanation, but they help provide additional evidence for the system design, implementation, and testing.

Appendix A: Demo Accounts

Role	Email	Purpose
Citizen / User	user@rukhsty.jo	Used to test citizen services such as application submission, tracking, payment, license viewing, and delivery request.
Admin	admin@rukhsty.jo	Used to manage users, applications, reports, and system data.
Security Officer	security.officer@rukhsty.jo	Used to review submitted applications and approve or reject them.
Medical Officer	medical.officer@rukhsty.jo	Used to record medical and vision test results.
Theory Officer	theory.officer@rukhsty.jo	Used to record theory exam results.
Practical Officer	practical.officer@rukhsty.jo	Used to record practical driving exam results.

Appendix B: Main System Workflow

The main workflow of RukhsTy starts when the citizen creates an account and submits a driving license application. The application is then sent to the security officer for review. After approval, the citizen books a medical or vision test. If the medical test is passed, the citizen books the theory exam. After passing the theory exam, the citizen books and completes the practical exam. When all stages are completed successfully, the system issues the digital driving license. The citizen can then view the license, verify it using QR code, pay through the mock eFAWATEERcom flow, and request mock Aramex delivery.